codefinity

Backend Development with Node.js and Express.js

Learn to build backend applications with Node.js and Express.js. Set up your environment and create your first Node.js apps. Work with the file system and build simple console tools. Use Express.js to build web apps, handle routes, and manage requests.

codefinity

Introduction to Node.js and Backend Development

codefinity

📖 **The Role of the Backend in Web Development**

The Backbone of Web Interactions

Welcome to the dynamic world of backend development, where we unveil the magic behind web experiences. While backend development spans many fields, this course will hone the backend's role in crafting interactive web experiences. In this chapter, we'll dive deeper into the specific aspects of backend development that pertain to the web environment.

Why Do We Need Backend Development?

Backend development is like the engine of a car. It powers everything behind the scenes, like storing data securely, doing calculations, and making sure everything works together smoothly. It's what makes apps and websites run properly, even though you don't see it directly.

Imagine you're using a social media app

1. **Data Storage**: The Backend stores your posts, photos, and info safely so you can access them anytime;
2. **Security**: It keeps your account safe by ensuring only you can access your stuff;
3. **Making Things Work**: When you post something, the backend figures out where to put it, who can see it, and how to show it to others;
4. **Notifications**: It sends notifications when someone likes your post or sends you a message;
5. **Search**: The Backend finds what you want when searching for friends or topics;
6. **Fast Responses**: It responds quickly when you want to see your feed, even when many people use the app.

The best part is that **YOU** can independently develop both the Frontend and Backend of your app using only JavaScript.

codefinity

📖 **What Is Node.js?**

The World of Node.js 🛠️

We will uncover why Node.js has become a rockstar in backend development. Even if you're not a tech wizard, we're here to unravel the mystery behind this powerful tool.

Node.js is a runtime environment that enables developers to create and execute applications with JavaScript. This language is typically associated with web browsers, but Node.js extends its use to server-side applications, which is useful.

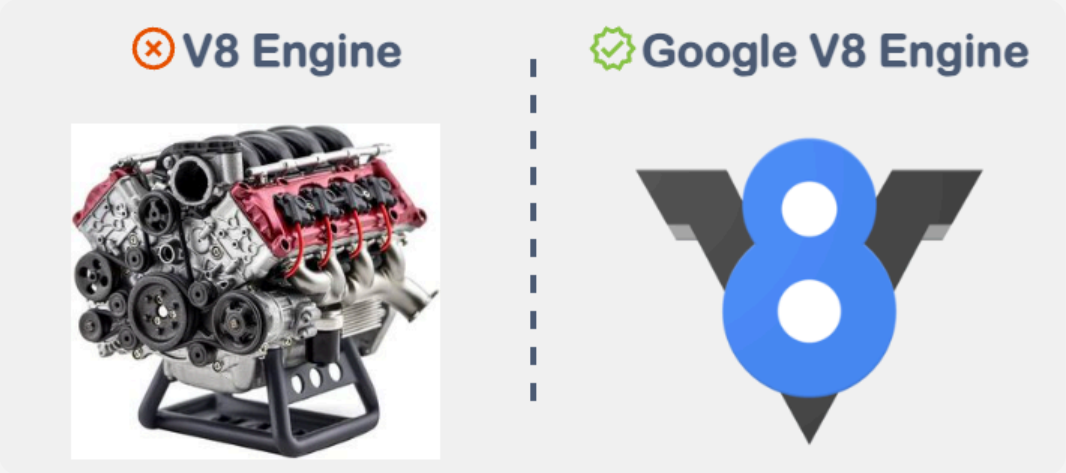
Node.js enables the use of JavaScript as a versatile language for both Frontend and Backend development, streamlining the process with a single language.

codefinity

🕮 Key Advantages of Using Node.js

Discover the reasons behind Node.js's prominence.

Harnessing the Power of the V8 Engine



Node.js is powered by Google's renowned V8 engine. Now, don't be misled by the term "engine" – we're not talking about cars here.

This exceptional engine compiles JavaScript code into highly efficient machine code, igniting the speed and responsiveness that characterize Node.js applications.

> **Note**
>
> **Machine code** is a language that computers understand directly. It consists of binary instructions that tell the computer's CPU what to do. In the context of Node.js, the V8 engine compiles JavaScript code into machine code, making it faster and more efficient for the computer to execute.

The Asynchronous Model and Event Loop

Node.js is a game-changer due to its unique architecture. Unlike traditional server-side environments that suffer from performance issues when handling multiple requests, Node.js utilizes an event-driven, non-blocking model to manage numerous connections simultaneously.

The result? Highly scalable applications that seamlessly handle extensive user interactions and requests.

Empowering Developers with NPM (Node Package Manager)

NPM, bundled with Node.js, enhances the development experience as a repository of numerous open-source libraries and tools. These resources are designed to boost the capabilities of Node.js applications, offering a range of pre-built solutions for improved functionality.

NPM also simplifies the complex task of managing library versions, freeing developers to focus on crafting exceptional application logic.

Node.js, with its V8 engine, asynchronous architecture, and vast collection of NPM packages, empowers developers to create lightning-fast and highly scalable web applications.

codefinity

📖 Installing Node.js

Now that we understand what Node.js is and its purpose, it's time to roll up our sleeves and get our hands on the real deal. As Node.js is a JavaScript runtime environment, installing it on your local machine is highly recommended.

System Requirements

Before diving in, let's quickly check the system requirements to ensure your computer is ready to handle Node.js: - 🪟 **Windows**: Windows 7 or later; - 🍏 **macOS**: macOS 10.12 (Sierra) or later; - 🐧 **Linux**: A modern Linux distribution with GLIBC version 2.17 or later, along with development tools (GCC, G++, etc.) for compiling native addons.

> **Note**
>
> Node.js and npm (Node Package Manager) are often installed together, as npm comes bundled with Node.js.

Installing Node.js

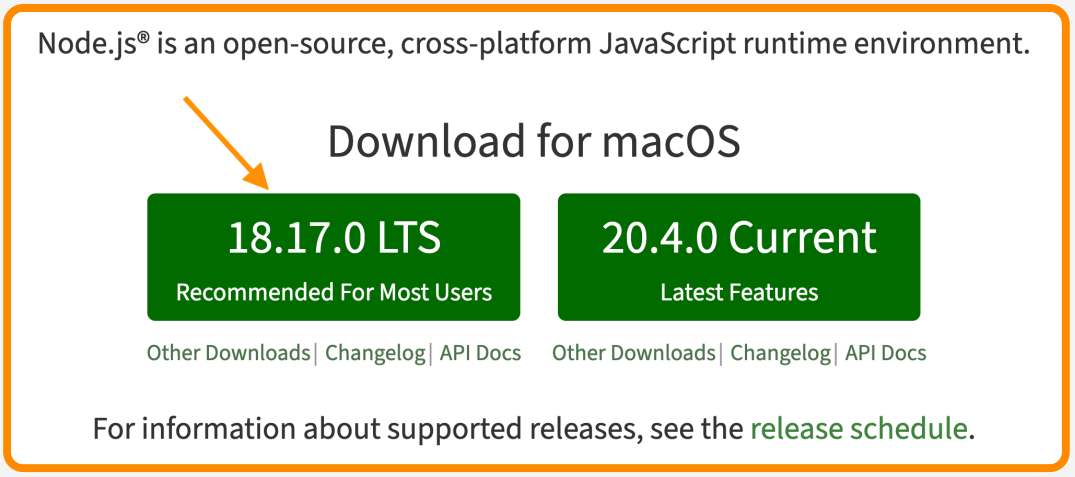
Here's a step-by-step guide on how to install Node.js on your machine:

Step 1: Visit the Official Website

Open the web browser and go to the official Node.js website at 📎 **https://nodejs.org/**.

Step 2: Download Node.js

Two main versions are available: LTS (Long-Term Support) and Current. We recommend installing the LTS version for most users, as it's stable and widely used.



1. Click on the "Downloads" tab on the Node.js website.
2. The website will automatically detect your operating system and recommend the appropriate LTS version. If not, select the LTS version from the options provided.
3. Click the "LTS" button to initiate the installer download for your operating system.

Step 3: Installation

codefinity

Windows Instructions

- Locate the downloaded Node.js installer (e.g., node-vXX.XX.X-x64.msi) on your computer and double-click it to begin the installation process;
- Follow the setup wizard's instructions:
    - Click "Next" to continue;
    - Accept the license agreement and click "Next";
    - Choose the destination folder for Node.js installation or stick with the default location. Click "Next";
    - Keep the default options, including npm, selected. Click "Next";
    - Choose whether to include the Node.js runtime in the system's PATH environment variable. Select "Automatically" for most cases. This allows you to run Node.js from any command prompt. Click "Next";
    - Click "Install" to start the installation;
    - Once the installation is complete, click "Finish" to exit the setup wizard.

Congratulations! You've successfully installed Node.js on your computer using the recommended LTS version. 🎉 In the next chapter, we'll dive into the fascinating Node.js ecosystem and learn how to create your first Node.js application.

codefinity

📖 Creating Your First Node.js App

🚀 Let's take our first step into the world of Node.js by building a simple application. In this app, we'll generate a random number between 1 and 100 and display it on the screen. This exercise will help you get acquainted with creating and running a Node.js application on **your local machine**.

> **Note**
>
> We highly recommend working locally on your machine for the best learning experience.

Follow these steps 📝 to create your first Node.js app:

1. Create a New Project Directory

Begin by creating a new directory for your project. It must be the empty folder somewhere on your machine. Then open this folder in your code editor. We will use 📎 **Visual Studio Code** to demonstrate examples. You can select any that is convenient for you.

2. Create the app.js File

Inside the created directory, create a new file named `app.js`. This is where we'll write the code for our Node.js application.

3. Write the Application Code

```
const randomNumber = Math.floor(Math.random() * 100) + 1;
console.log("Random Number: ", randomNumber);
```

This code uses JavaScript's built-in `Math.random()` function to generate a random number between 0 and 1. By multiplying it by 100 and adding 1, we ensure that the result falls between 1 and 100.

4. Run the Application

Open your terminal or command prompt and ensure you're still in the directory with the `app.js` file. Run the following command to execute the `app.js` file using Node.js:

```
node app.js
```

You should see the output displaying the randomly generated number:

```
Random Number: <some_random_number>
```

Real World Scenario 🌍

To further enhance your understanding, you can watch the following video, which demonstrates how to create and run the app in a real-world scenario:

Congratulations, you've successfully created and run your first Node.js application! 🎉 You've now taken your first step into the world of Node.js development.

codefinity

📖 Challenge: Build Your First Node App

🏆 Challenge

🎯 Goal

Prepare to begin a coding challenge to test your newfound knowledge of creating a simple Node.js application. The goal is to solidify your understanding of building a basic application using a single file and running it locally on the machine. This experience is crucial for your journey, as you'll frequently work locally in your development environment.

📋 Task

Create the Node.js application that calculates the total amount of money a user should pay for a given order. Here are the steps to complete the task:

1. Start by opening a new project directory where you'll create a file named `app.js`.
2. Inside the `app.js` file, set three predefined variables:
3. `ordererQuantity`: The number of tumblers the user ordered;
4. `pricePerTumbler`: The price of a single tumbler;
5. `message`: The message to be logged in the console.
6. Your main task is to calculate the total amount of money the user must pay for the order.
7. Once you've calculated the total cost, use the `console.log()` function to display the message and the estimated total amount.
8. After writing the code, open your terminal and run the application.
9. You should see the result in the console, displaying the total amount the user needs to pay.

```
// app.js
const ordererQuantity = 4;
const pricePerTumbler = 49;
const message = "You have to pay:";
console.log(message, ___);
```

codefinity

📖 **Key Takeaways**

Journey Recap

You've ventured into the world of backend dynamics, understanding its pivotal role in web orchestration. You've harnessed JavaScript and Node.js, unveiling their power.

Node.js's secrets are now yours, as you've embraced its asynchronous rhythm, Google V8 engine, and the NPM symphony for seamless library management.

With confidence, you've installed Node.js on your local machine, fusing the digital and tangible realms and sparking your creativity.

You've taken command of your coding domain, crafting your first Node.js creations – two apps that mark the beginning of your journey.

codefinity

Building Console Applications with Node.js

codefinity

🕮 Introduction to Console Applications in Node.js

Console Applications! No worries if it's new – we'll guide you through crafting your own programs that run on any Node.js-equipped machine. From your computer to your everyday electronic devices, you'll be able to command them all.

Discover What Awaits

- **Console Applications**: Understand what console applications are, their benefits, and how mastering them can impact your development journey;
- **FileSystem**: Navigate and manipulate files and directories, performing common file operations;
- **Command Line Interface (CLI)**: Learn command-line basics, create custom commands, and streamline your workflow;
- **Commander Module**: Use Node.js's Commander module to build feature-rich command-line interfaces;
- **Readline Module**: Enhance user interaction in the command line with the Readline module;
- **Practical Application**: Put your skills to the test by creating a console-based guessing game and an advanced directory inspection tool.

Ready to master console application development? Let's get started!

📖 **What Are Console Applications?**

Definition and Impact

Console applications, often called **command-line applications**, are versatile tools designed to operate within a text-based interface, such as a command prompt or terminal. In contrast to their graphical user interface (GUI) counterparts, console apps interact with users through text-based input and output, making them powerful assets in automation, efficient task execution, and system administration.

Versatility and Benefits

Console applications have a wide range of uses across different fields. System administrators depend on them for important duties such as handling files and configuring networks, while developers utilize their capabilities for batch processing, scripting, and creating utilities. They offer advantages such as increased productivity, speedy execution, compatibility with headless environments, and a straightforward path to automation.

Node.js - Your Console Craftsmanship Playground

Node.js stands out as an exceptional platform for crafting console applications. In fact, you've already created two console apps in the previous section.

codefinity

📖 Working with the File System in Node.js

The **FileSystem** module (`fs`) is a core module in Node.js, providing powerful capabilities for interacting with files programmatically. This module is helpful in various tasks, including managing configurations, handling data organization, and reading and writing file content.

📖 File Reading with fs.readFile

The `fs.readFile` method returns a promise that resolves with the file's contents. It allows for asynchronous file reading, making it suitable for reading text and binary files.

```
fs.readFile(path, options)
```

- `path` - the file path to read;
- `options` - an optional object specifying options like the encoding.

Imagine building a dynamic blogging platform. Here, the `fs.readFile` method steps onto the stage, quickly retrieving blog post content from a file.

Code Example: Reading Content

```
const fs = require('fs').promises;

fs.readFile('blog-post.txt', 'utf-8')
  .then(content => {
    console.log('Current content:', content);
  })
  .catch(err => {
    console.error('Error reading file:', err);
  });
```

Step by Step Explanation

🖋️ File Writing with fs.writeFile

The `fs.writeFile` method returns a promise that resolves when the file has been written. It is used to asynchronously write data to a file, which can be new or existing. It provides options for specifying the data, encoding, and file permissions.

```
fs.writeFile(file, data, options)
```

- `file` - the file path to write to;
- `data` - the data to write, which can be a string or a buffer;
- `options` - an optional object specifying options like the encoding and file mode.

Imagine we need to save a new user to the `user-db.json` file. Here `fs.writeFile` method ensures our words find their place.

Code Example: Writing User Data

```
const fs = require("fs").promises;

const newUser = {
  id: 1,
  username: "Nero",
  email: "arman@example.com",
};

const fileName = "user-db.json";

fs.writeFile(fileName, JSON.stringify(newUser), "utf-8")
  .then(() => {
    console.log("User information saved successfully.");
  })
  .catch((err) => {
    console.error("Error writing file:", err);
  });
```

Step by Step Explanation

📄 Extending with fs.appendFile

The `fs.appendFile` method returns a promise that resolves when the data has been appended to the file. It is used to asynchronously append data to an existing file, preserving its prior content.

```
fs.appendFile(file, data, options)
```

- `file` - the file path to which data will be appended;
- `data` - the data to append, which can be a string or a buffer;
- `options` - an optional object specifying options like the encoding and file mode.

Picture a bustling chat application recording conversations. As new messages flow, the `fs.appendFile` method adds new messages to the chat log while preserving the prior interactions.

Code Example: Appending Chat Messages

```
const fs = require("fs").promises;

const newMessage = "User2: Hello, how are you?";

fs.appendFile("chat.txt", newMessage + "\n")
  .then(() => {
    console.log("Message added to chat log.");
  })
  .catch((err) => {
    console.error("Error appending message:", err);
  });
```

Step by Step Explanation

> **Note**
>
> - `fs.writeFile` is used to completely replace the content of a file or create a new file;
> - `fs.appendFile` is used to add new data to the end of an existing file without overwriting what's already there.

🧐 Quiz Time

Let's gauge your understanding of FileSystem (`fs`) module concepts:

c<>definity

📖 Challenge: File System Operations

🏆 Challenge

🎯 Goal

Master the art of task management automation! Your mission is to develop an application that gathers tasks from one source, extracts their content, and integrates them into another file. Your solution should also handle any potential errors along the way.

📋 Task

Imagine you have two files: `tasks.txt`, which contains a list of existing tasks, and `new-task.txt`, which includes a single task that must be added to the `tasks.txt` file.

Follow these steps to complete the challenge and create the real deal on your machine: 1. **Prepare Your Workspace**: Start by creating a new folder on your machine and open it using your favorite code editor. 2. **Setup Tasks**: Create the `tasks.txt` file and populate it with the following tasks or use the provided 📎 **tasks.txt** file: - `Teach a goldfish 🐠 to play chess ♟️`; - `Build a sandcastle 🏰 in your living room 🛋️`; - `Create a song 🎶 using only sounds from nature 🌿`. 3. **Define New Task**: Create the `new-task.txt` file and insert the following task or use the provided 📎 **new-task.txt** file: - `Invent a new dance move and perform it in public. 💃🕺`. 4. **Main Script**: Craft the `app.js` file, which will serve as the heart of your application. - **Import fs Module**: Begin by importing the `fs` module to enable file handling within your application; - **Read Content**: Utilize the `readFile` function from the `fs` module to extract the content from the `new-task.txt` file. Be sure to implement `.then()` and `.catch()` to manage both success and error scenarios; - **Append Content**: Inside the `.then()` block, once the content is successfully read, employ the `appendFile` function to add the content to the `tasks.txt` file. Don't forget to append a newline character (`\n`) after the content. 5. **Run the Magic**: Save your `app.js` file and execute it using Node.js in the terminal with the command `node app`.

If you prefer to use the code editor below, *keep in mind that it does not recognize your files and will not show your progress.*

```
const fs = require("fs").___;

fs.___("new-task.txt", "utf-8")
  .then(___ => {
    return fs.___("tasks.txt", ___ + ___);
  })
  .___((error) => {
    console.log("Error:", error);
  });
```

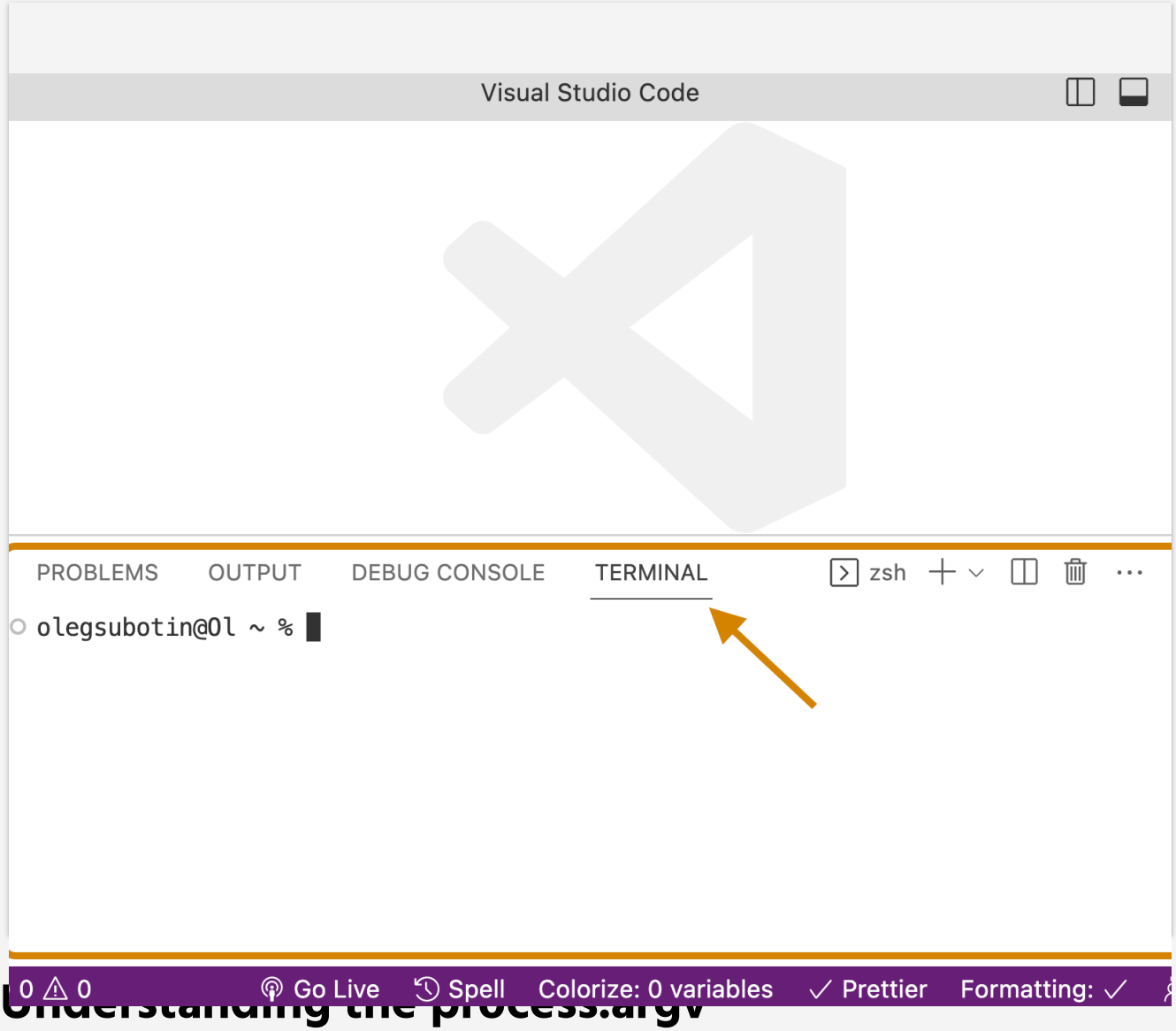
What Are CLI Applications?

Command Line Interface (CLI) applications are tools that allow users to communicate with software through command-line instructions. They offer speed, efficiency, and automation capabilities, making them ideal for various tasks.

Benefits of CLI Applications:

- **Quick Execution of Tasks**: CLI apps are lightning-fast when it comes to task execution. You can accomplish tasks with just a few keystrokes;
- **Automation Potential**: They are automation-friendly, enabling you to create scripts and automate repetitive tasks effortlessly;
- **Suitable for Server Environments**: CLI apps are well-suited for server environments, where graphical interfaces may not be available or practical.

In previous chapters, you might recall encountering the terminal while running Node apps. It is that CLI we are talking about.



Understanding the process.argv

When you fire up a Node.js script (by typing `node app`) in the command line, the `process.argv` array becomes your trusty sidekick. It carries the arguments you provide along with the command. This array is like a treasure chest with: - Element 0: The path to the Node.js executable; - Element 1: The path to the script being executed; - Elements 2 and beyond: Any additional arguments provided by the user.

You've got the theory, and now it's time to witness it in action. Launch the same Node script on your local machine and witness the magic.

Have you ever wondered where Node.js resides on your computer? Now you can find out with a single line of code:

```
console.log(process.argv);
```

CLI App Example

The true power lies in how we wield these arguments in the scripts. Behold an example script that calculates the sum of numbers we provide as arguments:

```
const args = process.argv.slice(2);
const sum = args.reduce((total, num) => total + parseFloat(num), 0);
console.log('Sum:', sum);
```

▶ Code Description

We're running the script and witnessing the magic unfold before our eyes.

codefinity

📖 Using the Commander Module for CLI Apps

Developing a command-line interface (CLI) with a module such as Commander can be more manageable. Although Node.js offers built-in mechanisms to handle command-line arguments, they can quickly become complicated to manage as the CLI expands. This is where Commander comes in handy, as it offers the following benefits: - **Streamlined CLI Development**: The Commander simplifies creating a CLI by minimizing the complexity, allowing us to focus on defining commands and their functions; - **Detailed Command and Option Descriptions**: With Commander, we can quickly provide descriptions for commands and options, enhancing the CLI's user-friendliness; - **Automatic Argument Parsing**: Commander automates the procedure of parsing command-line arguments, which minimizes the manual argument handling code we need to compose.

🔌 Installing the Commander Module

To begin with, we need to install the Commander module from NPM. But don't worry, the process is straightforward. Before we can start using the Commander module, we need to add it to our project. Just run the following command to install it:

```
npm install commander
```

🔧 Creating Commands and Options

With Commander, we can define commands, specify their behavior, and even provide descriptions. Additionally, we can explain options that modify the behavior of commands. Here's a sneak peek of what we can achieve with Commander:

Defining a Command

To define a command, use the `.command()` method of the program object. Here's the basic structure:

```
program.command('commandName [arguments]')
```

- `commandName` : This is the name of the command;
- `[arguments]` : These are optional arguments that the command accepts.

Adding Command Descriptions

We can provide a description for the command using the `.description()` method:

```
program.command('commandName [arguments]').description('Description of the command');
```

Handling Command Actions

Specify the action to be taken when the command is executed using the `.action()` method. This is where we define the logic associated with the command:

```
program.command('commandName [arguments]').action((...arguments) => {
  // Command logic goes here
});
```

Here's a complete example:

```
program
  .command('greet <name>')
  .description('Greet a person')
  .action((name) => {
    console.log(`Hello, ${name}!`);
  });
```

codefinity

Here's the code example from the video:

```
const { program } = require("commander");

program
  .version("1.0.0")
  .command("greet <name>")
  .description("Greet a person")
  .action((name) => {
    console.log(`Hello, ${name}!`);
  });

program.parse(process.argv);
```

▶ Code Description

👨‍💻 Try It Yourself!

Learning is best experienced through hands-on practice. Try running this code on your computer and watch the magic happen. Interact with the script and enjoy the excitement of creating unique greeting messages with the help of the Commander!

📖 Working with the Readline Module

👋 Getting Started with Readline

The Readline module is a built-in module in Node.js that simplifies reading input from the user in an interactive manner. It's often used to create prompts, collect user responses, and build interactive question-answer sessions in the command-line interface.

📖 Understanding the Readline Module

Before diving into a real-world example, let's explore the core concepts and methods provided by the Readline module.

Creating a Readline Interface

To use the Readline module, we typically start by creating a Readline interface that manages input and output streams. Here's how we create an interface:

```
const readline = require("readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});
```

- `const rl` represents the Readline interface;
- `readline.createInterface({ input: process.stdin, output: process.stdout })` sets up the interface to read from the standard input (`process.stdin`) and write to the standard output (`process.stdout`).

Collecting User Input

Once we have a Readline interface, we can use it to collect user input. The most common method for this purpose is `rl.question()`:

```
rl.question("Please enter your name: ", (name) => {
  // User input is available as `name`
  console.log(`Hello, ${name}!`);
  rl.close();
});
```

- `rl.question("Please enter your name: ", (name) => { ... })` prompts the user for input with the provided message;
- The callback function `(name) => { ... }` is executed when the user enters their response;
- `name` contains the user's input.

Managing the Interface

It's important to close the Readline interface when we're done with it. We can do this using `rl.close()`.

🔧 Creating an Interactive Prompt

Now that we've covered the basics let's put our knowledge to use and create a fun command-line fortune teller. Users will enter their names, and the application will generate a random fortune message.

Here's the code example from the video:

codefinity

```
const readline = require("readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

const fortunes = [
  "You will find unexpected joy in the little things.",
  "A new opportunity will open doors for you.",
  "Adventure is just around the corner.",
  "Embrace change, and good things will follow.",
  "Patience will bring you great rewards.",
];

rl.question("Welcome to the Fortune Teller! What's your name? ",
(name) => {
  const randomIndex = Math.floor(Math.random() * fortunes.length);
  const randomFortune = fortunes[randomIndex];

  console.log(`Hello, name!Yourfortunetoday:{name}! Your fortune
today: name!Yourfortunetoday:{randomFortune}`);

  rl.close();
});
```

In this example, we've applied the above concepts to create an interactive command-line application. Users are prompted for input, and the application generates random responses. The Readline module simplifies user interaction in the command-line interface, making it ideal for creating interactive CLI applications.

▶ Code Description

codefinity

📖 Building a Guessing Game Console App

In this chapter, get ready to level up your console application skills as we dive into the creation of an exciting 🎮 **Guess the Number** 🔢 game app. This interactive game will challenge players to exercise their intuition by guessing a randomly generated number within a predefined range. Along the way, we'll unravel the mysteries of fundamental concepts such as - 🎲 Random number generation; - ✅ Input validation; - 🤝 User interaction; - 💾 Even saving game results to a file.

🏆 Challenge Awaits

Imagine delving into an app that promises excitement and intrigue. Players are invited to guess a mysteriously chosen number within a predefined range. The app offers instant feedback on each guess and a meticulous tally of the attempts made.

This hands-on example is your chance to refine your skills in building CLI Apps and showcases the art of crafting interactive programs.

🚀 Resulting App

See the magic in action! Below is a GIF illustrating the exciting Guess the Number Game app that you'll be crafting:

Building a Guessing Game Console App

You're presented with two paths. The first option beckons you to embark on the journey without assistance, while the second offers a helpful guide to ensure your success. Whether you dive in boldly or follow the structured guide, you're in for a fascinating experience that will leave you with a functional and captivating console app.

Masterplan

Step 1: Setup and Initializations

Prepare the canvas by creating a new directory and a file named `app.js`. Within this file, we conjure the necessary modules:

```
const readline = require('readline');
const fs = require('fs').promises;
```

Create a Readline interface:

```
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});
```

▶ Code Description

Explanation: We import the necessary modules: `readline` for user interaction and `fs.promises` for file operations. Then, we create a `Readline` interface named `rl` to handle input and output.

Step 2: Define Game Parameters

Set the minimum and maximum numbers:

```
const minNumber = 1;
const maxNumber = 100;
```

Generate the secret number:

```
const secretNumber =
  Math.floor(Math.random() * (maxNumber - minNumber + 1)) + minNumber;
```

▶ Code Description

Initialize the attempts counter:

```
let attempts = 0;
```

Explanation: We define the range of numbers (`minNumber` and `maxNumber`) within which the secret number will be generated. The secret number is randomly generated using `Math.random()` and assigned to `secretNumber`. The `attempts` variable is initialized to keep track of the user's attempts.

Step 3: Define Utility Functions

Equip with a utility function for impeccable validation:

```
function isValidGuess(guess) {
  return !isNaN(guess)
    && guess >= minNumber
    && guess <= maxNumber;
}
```

▶ Code Description

Explanation: The `isValidGuess` function checks whether the user's guess is a valid number within the specified range (`minNumber` to `maxNumber`).

Step 4: Game Logic

The game's core mechanics through the `playGame` function:

```
function playGame() {
  rl.question(`Guess a number between minNumberand{minNumber} and
minNumberand{maxNumber}: `, guess => {
    if (isValidGuess(guess)) {
      attempts++;
      const guessNumber = parseInt(guess);

      if (guessNumber === secretNumber) {
        console.log(`Congratulations! You guessed the number in
attemptsattempts.‘);saveGameResult(‘Playerwonin{attempts} attempts.`);
        saveGameResult(`Player won in
attemptsattempts.‘);saveGameResult(‘Playerwonin{attempts} attempts.`);
        rl.close();
      } else if (guessNumber < secretNumber) {
        console.log('Try higher.');
        playGame();
      } else {
        console.log('Try lower.');
        playGame();
      }
    } else {
      console.log('Please enter a valid number within the specified
range.');
      playGame();
    }
  });
}
```

▶ Code Description

Explanation: The `playGame` function forms the core game loop. It uses `rl.question` to prompt the user for a guess. If the guess is valid, the function checks if the guess matches the secret number. If not, it provides feedback to the user and continues the game loop recursively.

Step 5: Save Game Result

Implement the logic of saving the user's game achievements into the `game_results.txt` file.

```
async function saveGameResult(result) {
  try {
    await fs.appendFile('game_results.txt', `${result}\n`);
    console.log('Game result saved.');
  } catch (err) {
    console.log('Failed to save game result.');
  }
}
```

▶ Code Description

Explanation: The `saveGameResult` function uses `fs.promises` to append the game result to a file named `game_results.txt`. It provides feedback on the success or failure of saving the result.

Step 6: Start the Game

Create the welcome message and launch the game:

```
console.log('Welcome to the Guess the Number game!');
playGame();
```

Explanation: We display a welcome message and start the game loop by calling the `playGame` function.

🎉 Conclusion: Victory Lap

By developing the **Guess the Number Game App**, you've gained valuable experience in designing interactive and engaging console applications. This example showcases the fusion of user input, random number generation, validation, and file manipulation, resulting in a compelling gaming experience.

👨‍💻 Full App Code

```
const readline = require("readline");
const fs = require("fs").promises;

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

const minNumber = 1;
const maxNumber = 100;

const secretNumber =
  Math.floor(Math.random() * (maxNumber - minNumber + 1)) + minNumber;
let attempts = 0;

function isValidGuess(guess) {
  return !isNaN(guess) && guess >= minNumber && guess <= maxNumber;
}

function playGame() {
  rl.question(
    `Guess a number between minNumberand{minNumber} and
minNumberand{maxNumber}: `,
    (guess) => {
      if (isValidGuess(guess)) {
        attempts++;
        const guessNumber = parseInt(guess);

        if (guessNumber === secretNumber) {
          console.log(
            `Congratulations! You guessed the number in
attemptsattempts.‘);saveGameResult(‘Playerwonin{attempts} attempts.`
```

```
          );
          saveGameResult(`Player won in
attemptsattempts.‘);saveGameResult(‘Playerwonin{attempts} attempts.`);
          rl.close();
        } else if (guessNumber < secretNumber) {
          console.log("Try higher.");
          playGame();
        } else {
          console.log("Try lower.");
          playGame();
        }
      } else {
        console.log("Please enter a valid number within the specified
range.");
        playGame();
      }
    }
  );
}

async function saveGameResult(result) {
  try {
    await fs.appendFile("game_results.txt", `${result}\n`);
    console.log("Game result saved.");
  } catch (err) {
    console.log("Failed to save game result.");
  }
}

console.log("Welcome to the Guess the Number game!");
playGame();
```

codefinity

▶ Code Description

codefinity

📖 Managing Directories in Node.js

We've learned many file manipulation techniques throughout our exploration of the FileSystem (`fs`) module. But directories are more than places to store files - they offer opportunities for organization, data analysis, and more.

In this chapter, we'll dive into directory manipulation and guide you on navigating directories, gathering vital statistics, processing directories, and creating a script to analyze and display directory contents.

📂 Navigating Directories with fs.readdir

The `fs.readdir` method asynchronously reads the contents of a directory. It returns an array of filenames. This method can be beneficial for task listing files in a folder.

Consider a scenario where we deal with extensive legal contracts, briefs, and other documents related to different cases and clients. We could create a system that extracts and lists the names of the files within each client's folder.

Code Example: Read the file names of a directory

```
const fs = require("fs").promises;

async function listDirectoryContents(directoryPath) {
  try {
    const files = await fs.readdir(directoryPath);
    console.log("Directory Contents:", files);
  } catch (err) {
    console.error("Error reading directory:", err.message);
  }
}

listDirectoryContents("./docs");
```

Step by Step Explanation

📊 Obtaining Directories Statistics with fs.stat

Directories house files and hold valuable information about each file's attributes.

The `fs.stat` method asynchronously retrieves the stats of a file or directory. These stats include file size, permissions, timestamps, and more.

Let's obtain the statistics of each folder inside the docs folder.

Code Example: Get directories names and statistics

```
const fs = require("fs").promises;

async function processDirectoryContents(directoryPath) {
  try {
    const files = await fs.readdir(directoryPath);

    const fileStatsPromises = files.map(async (file) => {
      const filePath =
`directoryPath/{directoryPath}/directoryPath/{file}`;
      const stats = await fs.stat(filePath);
      return { name: file, stats };
    });

    const fileStats = await Promise.all(fileStatsPromises);
    console.log("Detailed File Information:", fileStats);
  } catch (err) {
    console.error("Error processing directory contents:",
err.message);
  }
}

processDirectoryContents("./docs");
```

Step by Step Explanation

🧐 Quiz Time

Let's put your knowledge of the FileSystem (`fs`) module to the test with a few questions related to directory manipulation.

codefinity

📖 Directory Inspection Tool

This chapter presents you with a challenge: creating an advanced console app named **DirInspect Pro**. This app will empower you to thoroughly analyze any directory and gain insightful statistics about its files and subdirectories.

🏆 Challenge Awaits

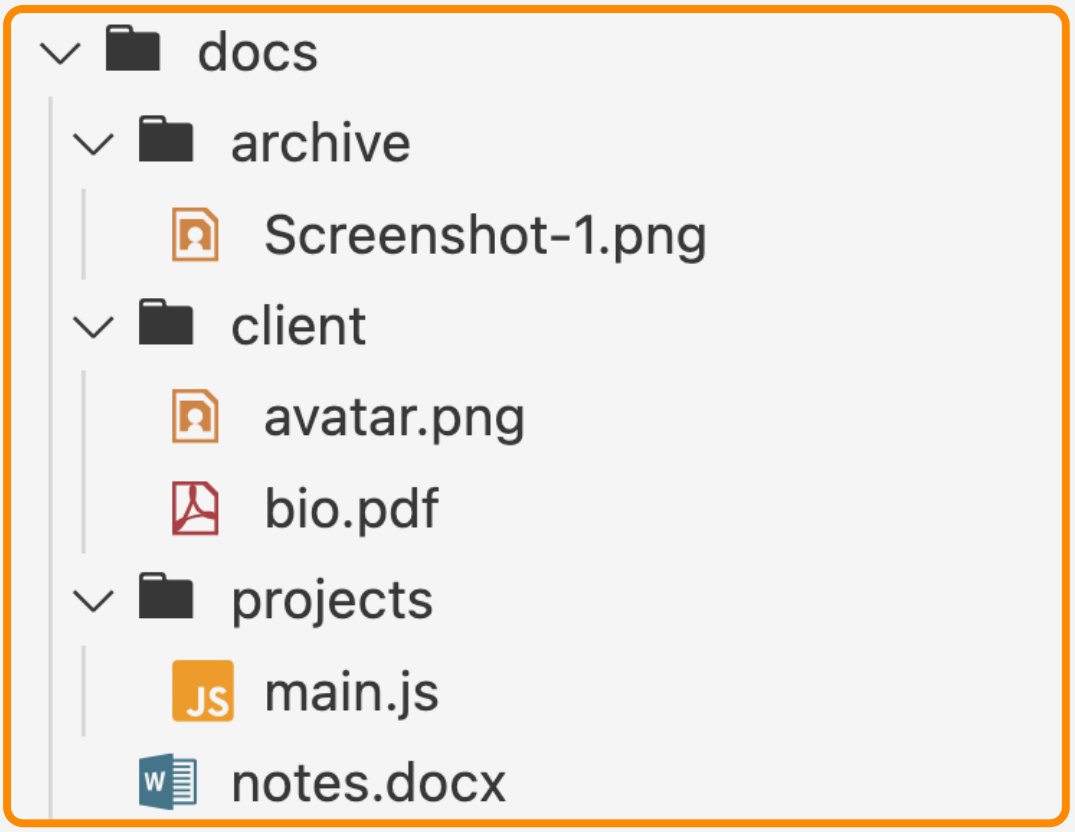
Picture a scenario where you must navigate a labyrinth of folders containing crucial files and data. DirInspect Pro is your ally in this journey, providing comprehensive insights into the directory's structure and contents.

🚀 The Resulting App

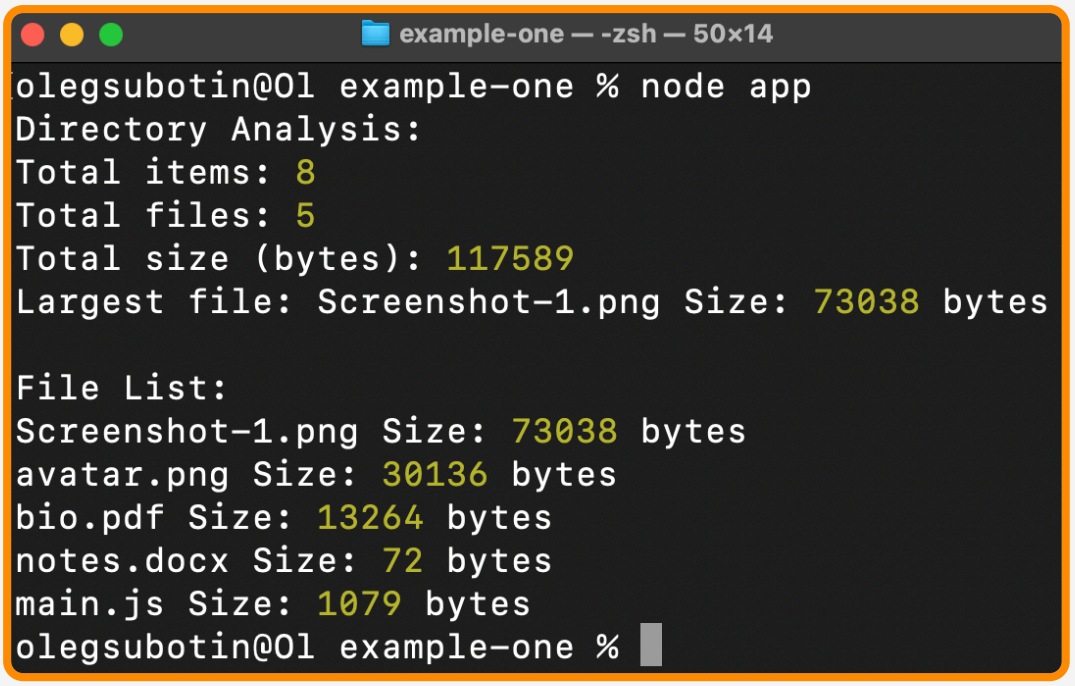
Get ready to wield DirInspect Pro's capabilities. The app will furnish you with critical information, such as - The total number of items; - The aggregate size of all items; - The largest file's name and size; - The detailed list of individual file names and sizes.

codefinity

App in Action



DirInspect Pro's Output:



Two Paths to Choose

You have two paths ahead. - The first is to tackle this challenge head-on, honing your skills without guidance; - The second is to follow a helpful guide that guarantees your success.

Whichever path you choose, you're in for a rewarding journey culminating in creating a captivating and functional console app.

Masterplan

Step 1: Import Required Modules

To embark on this adventure, you'll need the right tools. Begin by importing two key modules: `fs.promises` to manage the file system asynchronously and `path` to handle file paths effectively.

```
const fs = require("fs").promises;
const path = require("path");
```

▶ Code Description

Step 2: Define getStats Function

The asynchronous function, `getStats`, takes a file or directory path as an argument and attempts to retrieve its statistics using `fs.stat`. - If successful, it returns the statistics; - If an error occurs, it logs an error message and returns `null`.

```
async function getStats(filePath) {
  try {
    const stats = await fs.stat(filePath);
    return stats;
  } catch (err) {
    console.error("Error getting stats:", err.message);
    return null;
  }
}
```

▶ Code Description

Step 3: Define analyzeFile Function

The `analyzeFile` function uses the `getStats` function to obtain statistics for a file. If statistics are available (not `null`), it returns an object containing the file's name (extracted using `path.basename`) and its size.

```
async function analyzeFile(filePath) {
  const stats = await getStats(filePath);
  if (!stats) return null;

  return {
    name: path.basename(filePath),
    size: stats.size,
  };
}
```

▶ Code Description

Step 4: Define analyzeDirectory Function

The `analyzeDirectory` function takes a directory path as an argument and comprehensively analyzes its contents. - It begins by reading the items within the directory using `fs.readdir` and then iterates through each item; - For each item, it constructs the full path using `path.join` and retrieves its statistics using the `getStats` function; - If the `stats` indicate that the item is a file, it updates file-related statistics; - If the item is a subdirectory, it recursively calls the `analyzeDirectory` function to analyze its contents and then aggregates the statistics.

```
async function analyzeDirectory(directoryPath) {
  try {
    const items = await fs.readdir(directoryPath);
    let totalItems = 0;
    let totalFiles = 0;
    let totalSize = 0;
    let largestFile = { name: "", size: 0 };
    let fileList = [];

    for (const item of items) {
      const itemPath = path.join(directoryPath, item);
      const stats = await getStats(itemPath);
      if (!stats) continue;

      totalItems++;

      if (stats.isFile()) {
        totalFiles++;
        totalSize += stats.size;

        if (stats.size > largestFile.size) {
          largestFile = { name: item, size: stats.size };
        }

        fileList.push({ name: item, size: stats.size });
      } else if (stats.isDirectory()) {
        const subDirectoryStats = await analyzeDirectory(itemPath);
        totalItems += subDirectoryStats.totalItems;
        totalFiles += subDirectoryStats.totalFiles;
        totalSize += subDirectoryStats.totalSize;

        if (subDirectoryStats.largestFile.size > largestFile.size) {
```

```
            largestFile = subDirectoryStats.largestFile;
          }

          fileList = fileList.concat(subDirectoryStats.fileList);
        }
      }

      return {
        totalItems,
        totalFiles,
        totalSize,
        largestFile,
        fileList,
      };
    } catch (err) {
      console.error("Error analyzing directory contents:", err.message);
      return {
        totalItems: 0,
        totalFiles: 0,
        totalSize: 0,
        largestFile: { name: "", size: 0 },
        fileList: [],
      };
    }
}
```

▶ Code Description

Step 5: Define main Function and Invoke

codefinity

The `main` function is the entry point of the script. It specifies the directory path to analyze (in this case, `./docs`), calls the `analyzeDirectory` function to obtain the statistics of the directory and its contents, and then outputs the collected information. The function prints out - The **total number of items**; - The **total number of files**; - The **total size**; - The **details about the largest file**; - The **list of files in the directory**.

```
async function main() {
  const directoryPath = "./docs";
  const directoryStats = await analyzeDirectory(directoryPath);

  console.log("Directory Analysis:");
  console.log("Total items:", directoryStats.totalItems);
  console.log("Total files:", directoryStats.totalFiles);
  console.log("Total size (bytes):", directoryStats.totalSize);
  console.log(
    "Largest file:",
    directoryStats.largestFile.name,
    "Size:",
    directoryStats.largestFile.size,
    "bytes"
  );

  console.log("\nFile List:");
  for (const file of directoryStats.fileList) {
    console.log(file.name, "Size:", file.size, "bytes");
  }
}

main();
```

▶ Code Description

🎉 Conclusion: Mastered Skills

With DirInspect Pro, you've mastered the art of analyzing directories like a pro. This console app showcases your ability to extract file statistics, handle errors seamlessly, and unveil meaningful insights about files and subdirectories within a specified directory.

👨‍💻 Full App Code

```
const fs = require("fs").promises;
const path = require("path");

async function getStats(filePath) {
  try {
    const stats = await fs.stat(filePath);
    return stats;
  } catch (err) {
    console.error("Error getting stats:", err.message);
    return null;
  }
}

async function analyzeFile(filePath) {
  const stats = await getStats(filePath);
  if (!stats) return null;

  return {
    name: path.basename(filePath),
    size: stats.size,
  };
}

async function analyzeDirectory(directoryPath) {
  try {
    const items = await fs.readdir(directoryPath);

    let totalItems = 0;
    let totalFiles = 0;
    let totalSize = 0;
    let largestFile = { name: "", size: 0 };
    let fileList = [];
```

```
    for (const item of items) {
      const itemPath = path.join(directoryPath, item);
      const stats = await getStats(itemPath);
      if (!stats) continue;

      totalItems++;

      if (stats.isFile()) {
        totalFiles++;
        totalSize += stats.size;

        if (stats.size > largestFile.size) {
          largestFile = { name: item, size: stats.size };
        }

        fileList.push({ name: item, size: stats.size });
      } else if (stats.isDirectory()) {
        const subDirectoryStats = await analyzeDirectory(itemPath);
        totalItems += subDirectoryStats.totalItems;
        totalFiles += subDirectoryStats.totalFiles;
        totalSize += subDirectoryStats.totalSize;

        if (subDirectoryStats.largestFile.size > largestFile.size) {
          largestFile = subDirectoryStats.largestFile;
        }

        fileList = fileList.concat(subDirectoryStats.fileList);
      }
    }

    return {
      totalItems,
```

```
      totalFiles,
      totalSize,
      largestFile,
      fileList,
    };
  } catch (err) {
    console.error("Error analyzing directory contents:", err.message);
    return {
      totalItems: 0,
      totalFiles: 0,
      totalSize: 0,
      largestFile: { name: "", size: 0 },
      fileList: [],
    };
  }
}

async function main() {
  const directoryPath = "./docs";

  const directoryStats = await analyzeDirectory(directoryPath);

  console.log("Directory Analysis:");
  console.log("Total items:", directoryStats.totalItems);
  console.log("Total files:", directoryStats.totalFiles);
  console.log("Total size (bytes):", directoryStats.totalSize);
  console.log(
    "Largest file:",
    directoryStats.largestFile.name,
    "Size:",
    directoryStats.largestFile.size,
    "bytes"
  );
```

```
  console.log("\nFile List:");
  for (const file of directoryStats.fileList) {
    console.log(file.name, "Size:", file.size, "bytes");
  }
}

main();
```

codefinity

📖 Summary of Console Applications in Node.js

📚 Key Takeaways

- 👨‍💻 You've learned how to create interactive command-line applications using Node.js;
- 📝 Understanding command-line arguments, structured interfaces, and user input handling;
- 🎮 Building engaging games and practical utilities with interactive elements;
- 📊 Developing file management tools and directory analysis scripts;
- 🚀 Gaining hands-on experience in designing and coding console applications.

🔍 Practical Applications

The skills you've gained in this section have practical applications in various domains, including software development, system administration, data processing, and more. You can create custom tools, automate tasks, and enhance your productivity through well-designed console applications.

We are grateful for your unwavering dedication and enthusiasm throughout this learning journey. As you continue on your path of learning, exploration, and application, remember that the skills you've acquired are keys to unlocking endless possibilities.

codefinity

Developing Web Applications with Express.js

c<>definity

📖 Introduction to Express.js in Web Development

In this section, we'll put Express.js in the spotlight, making it the star of our journey. Get ready to: - 🚰 Discover how even the simplest water pipelines can illuminate fundamental concepts; - 👮 Discover how a 'postman' can be your secret ally in testing your apps and more; - 📡 Acquaint yourself with the term HTTP and realize you use it daily; - 📚 Get a fresh perspective on visiting the library and how it relates to web communication; - 🔀 Witness the magical transformation of your computer into a responsive server powerhouse.

Discover What Awaits 🕵️:

- 🚀 Discover the Express.js Framework;
- 🛠️ Learn How to Set Up an Express.js App;
- 🌐 Understand the Basics of HTTP Requests;
- 🚧 Create Custom Routes for Your Application;
- ⚙️ Explore Built-in and Custom Middleware.

It's crucial to emphasize that this section lays the groundwork, arming you with vital knowledge to breathe life into your app ideas. Think of it as the launchpad propelling us into the next section, where we'll dive headfirst into building APIs from scratch.

codefinity

🕮 Why Use Express.js for Web Development?

Express.js is a framework that enhances Node.js for web development, providing many advantages over plain Node.js.

Benefits of using Express.js over plain Node.js

Streamlined Routing

Express.js simplifies the intricate process of routing by allowing developers to define routes for different HTTP methods and URL patterns.

Flexible Middleware

One of Express.js's greatest assets is its middleware system. It enables the injection of modular functions at various stages of the request-response cycle. This flexibility facilitates crucial tasks like authentication, logging, and error handling. As a result, we can design secure and efficient web apps.

Serving Static Files

In addition to its server logic capabilities, Express.js excels at serving static files such as HTML, CSS, and JavaScript. This versatility makes it an ideal choice for the development of full-stack applications.

💡 Where Express.js Shines

Express.js is the preferred option for a diverse range of web development scenarios, including: - **Web Applications**: Whether it's a basic project or a complex application, Express.js streamlines the development process; - **API Development**: When building RESTful APIs to provide data for web and mobile apps, Express.js is a standout performer; - **Real-time Applications**: By teaming up with WebSocket libraries such as Socket.io, Express.js can fuel real-time apps like chat apps and online gaming platforms; - **Microservices Architecture**: In microservices-based architectures, Express.js can serve as the bedrock for individual microservices.

> **Note**
>
> A REST API, also called a RESTful API, is a type of application programming interface (API) that adheres to the principles of the REST architectural style. It enables communication with RESTful web services, where 'REST' stands for representational state transfer.

codefinity

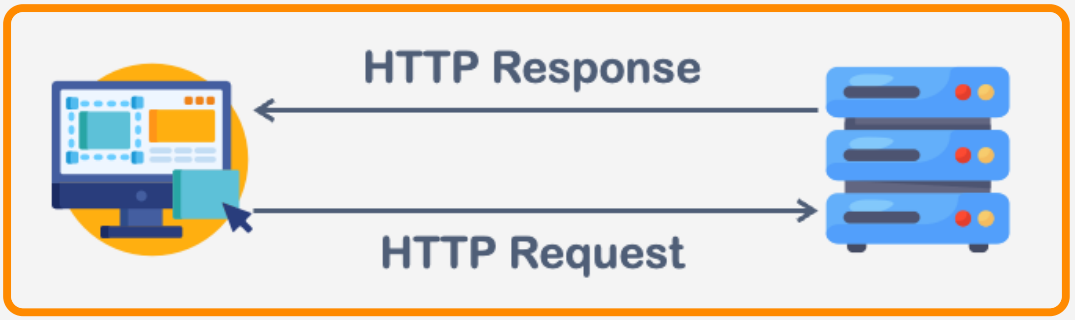
📖 Understanding HTTP Requests

Before we dive deeper into developing web applications, let's take a moment to revisit some crucial theory that underpins our understanding of what we're about to build and why.

🔄 Understanding the HTTP Request-Response

HTTP (Hypertext Transfer Protocol) is the foundation of data communication between a client (typically a web browser or an application) and a server. This protocol enables clients to request and receive server resources.

Request-Response Model: HTTP operates on a request-response model. Imagine a client sending an HTTP request to a server, and in return, the server promptly furnishes an HTTP response containing the requested data.



🏛️ Real-World Analogy (Visiting a Library)

To make this concept relatable, let's draw an analogy with a real-world scenario: visiting a library.

Client-Server Communication

As the client, you are in one room (the library), while the librarian at the front desk is in another room (the server). Just as in web communication, you can't directly access the books (resources); you need to communicate with the librarian (server) to get what you want.

Request-Response Model

Requesting a book isn't as simple as picking it off the shelf; there's a structured process. You approach the librarian and ask for a specific book - this mirrors an HTTP request.

Server Response

The librarian (server) takes your request seriously. They search the shelves (the server processes your request), locate the book, and hand it over to you. This action mirrors the server sending you an HTTP response containing the requested data (the book).

Communication Flow

The interaction between you and the librarian continues as long as you need more books. With each book you request, the librarian retrieves it and hands it to you. Similarly, in HTTP, the client can make multiple requests, and the server responds to each one.

> **Note**
>
> In the upcoming chapters, we will learn how to become the librarians of the web world, handling requests and serving resources.

📤 Types of HTTP Requests

HTTP defines several request methods, each with a specific purpose: - **GET**: Use it to retrieve data from a specified resource. It shouldn't have any side effects on the server. For example, fetching a webpage, an image, or data from an API; - **POST**: Use it to submit data to be processed at a specified resource. It can create a new resource or update an existing one; - **PUT**: Employ it to update a current resource with new data. Unlike POST, which can create new resources, PUT is idempotent, meaning the same operation can be repeated without changing the result; - **DELETE**: Request the removal of a resource. For instance, it's like deleting a user account or a file; - **PATCH**: Use it to apply partial modifications to a resource. It's often used for updating specific fields of an existing resource.

codefinity

📖 Setting Up an Express.js Application

Let's create our first backend app with Express.js. Are you ready to get started?

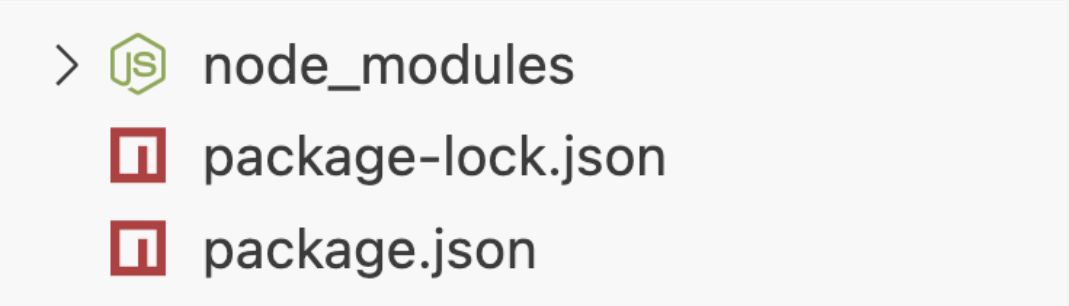
Installing Express.js

Create a new directory for the app, and open the folder in the code editor. We are ready to start. In the terminal, run this command:

```
npm install express
```

It's like ordering Express.js from a virtual app store, and npm is our delivery service.

As a result, we get such file-folder structure of our app:



Basic project structure:

- `node_modules` - Contains installed packages;
- `package.json` and `package-lock.json` - List project dependencies and scripts;
- `app.js` or `index.js` - Entry point for the Express application. We create it manually by ourselves.

🏗️ Build First Express App

Create a simple web server using Node.js and the Express.js framework. Follow the following steps:

Step 1: Import Express

As library we need firstly to import it to our file:

```
const express = require('express');
```

Step 2: Creating an Express Application Instance

We create an instance of the Express application. This `app` variable will be used to configure and define the web server's behavior.

```
const app = express();
```

Step 3: Set the Port

We define the port number that our server will listen on. In this case, it's set to 3000, but we can choose any available port number.

```
const port = 3000;
```

Step 4: Defining a Route

We set up a route for handling HTTP GET requests to the root URL (`/`). When a client (typically a web browser) accesses the server's root URL, it responds with `Hello, World!` .

```
app.get('/', (req, res) => {
  res.send('Hello, World!');
});
```

- `app.get('/')` - This defines a route for handling GET requests to the root path (`/`). We can define routes for different HTTP methods (GET, POST, PUT, DELETE, etc.);
- `(req, res) => { ... }` - This is a callback function that gets executed when a client makes a GET request to the specified route. It takes two arguments: `req` (the request object) and `res` (the response object). In this case, it simply sends the `Hello, World!` text as the response.

Step 5: Start the Server

Let's start the server and make it listen on the specified port (in our case, port 3000). When the server is successfully started, it logs a message to the console, indicating which port it is listening on.

```
app.listen(port, () => {
  console.log(`Server is running on port ${port}`);
});
```

`app.listen(port, ...)` - This method starts the server and listens on the specified port. The second argument is a callback function that gets executed once the server is up and running.

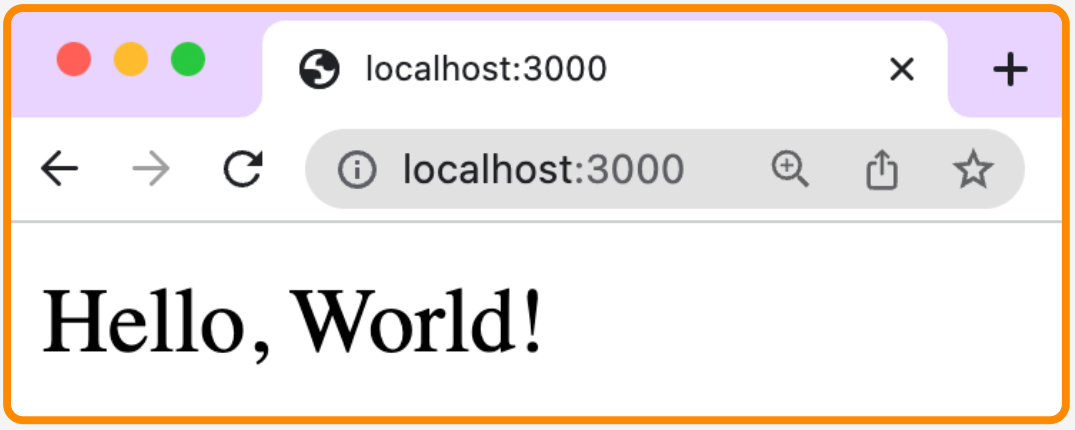
Step 6: Run the App

We run the app in the terminal using `node` command.

```
node index
```

🌐 After running the script

Our server will be running, and we can access it by opening a web browser and navigating to `http://localhost:3000`. You should see `Hello, World!` displayed in your browser.



codefinity

📖 Creating and Managing Routes in Express.js

Let's explore these practical examples of creating routes in Express.js in more detail.

🌟 Basic Routes

Let's create a simple Express.js application with routes for different HTTP methods. Each route handles a specific HTTP request and sends a response back to the client.

codefinity

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('This is the homepage.');
});

app.post('/users', (req, res) => {
  // Handle user creation logic here.
  res.send('User created.');
});

app.put('/users/:id', (req, res) => {
  // Handle user update logic here.
  res.send(`User ${req.params.id} updated.`);
});

app.delete('/users/:id', (req, res) => {
  // Handle user deletion logic here.
  res.send(`User ${req.params.id} deleted.`);
});

app.listen(3000, () => {
  console.log('Server is running on port 3000.');
});
```

- **GET Route**: This route responds to a GET request at the root URL (`/`). Users who access the application's homepage will receive the message "This is the homepage.";
- **POST Route**: This route responds to a POST request at the `/users` URL. It's typically used for creating new user records. The route includes a comment for handling user creation logic, which can be replaced with the actual user creation logic;
- **PUT Route**: This route responds to a PUT request at the `/users/:id` URL. It's designed for updating user records based on the `:id` parameter. Like the POST route, it includes a comment handling user update logic;
- **DELETE Route**: This route responds to a DELETE request at the `/users/:id` URL. It's used for deleting user records based on the `:id` parameter. As with the other routes, it includes a comment for handling user deletion logic.

> **Note**
>
> If you're uncertain about the URL or would like to review the concept, please refer to the following article: 📎 URL vs URI vs URN

🌟 Handle URL Parameters

Let's focus on creating a route that handles URL parameters. URL parameters are dynamic parts of a URL that can be used to pass data to the server. Here's how this example works:

```
app.get('/users/:id', (req, res) => {
  const userId = req.params.id;
  // Fetch user data based on `userId`.
  res.send(`User details for ID ${userId}.`);
});
```

- We define a route using `app.get('/users/:id', ...)`, where `:id` is a URL parameter. This means that when a user accesses a URL like `/users/123`, the `req.params.id` variable will contain the value `123`;
- Inside the route handler, we access the `:id` parameter using `req.params.id`;
- You can replace the comment with logic to fetch user data based on the `userId` obtained from the URL parameter;
- Finally, we respond to the client with information about the user identified by the `:id` parameter.

🌟 Route Order and Prioritization

The order in which routes are defined matters because Express uses a first-match-wins strategy. The first matching route will be executed. In this example, we consider a scenario where you have both a specific and a catch-all route:

```
app.get('/users/:id', (req, res) => {
  const userId = req.params.id;
  // Fetch user data based on `userId`.
  res.send(`User details for ID ${userId}.`);
});

app.get('/users/*', (req, res) => {
  // Handle generic user-related requests.
  res.send('User-related request.');
});
```

- The first route, `app.get('/users/:id', ...)`, is a specific route that responds to URLs like `/users/123`. It will handle requests with a specific user ID;
- The second route, `app.get('/users/*', ...)`, is a catch-all route for generic user-related requests. It uses a wildcard (`*`) to match any URL that starts with `/users/`, such as `/users/settings`.

In this scenario, the order is crucial. The specific route should be defined before the catch-all route to ensure it takes precedence. If you define the catch-all route first, it will match all URLs starting with `/users/`, and the specific route won't have a chance to handle requests with a user ID.

codefinity

🕮 Testing APIs with Postman

Testing Express.js routes is a fundamental skill for developers working with APIs. To do this effectively, we need specialized software capable of sending various types of HTTP requests and inspecting server responses. Enter Postman, a powerful API testing tool that simplifies this process. In this chapter, we'll guide you through using Postman to test your Express.js routes step by step.

Step 1: Install Postman

If you haven't already, download and install Postman from Download Postman.

Step 2: Launch Postman

Open the Postman application on your computer.

Step 3: Create a New Request

1. Click on the "New" button located in the top-left corner;
2. Choose "Request" to create a new request.

Step 4: Name Your Request

Give your request a meaningful name. For instance, you can name it "GET Homepage."

Step 5: Select HTTP Method

In the request editor, select the appropriate HTTP method you want to test (e.g., GET, POST, PUT, DELETE) from the dropdown menu.

Step 6: Enter Request URL

1. In the URL field, enter the URL of your Express.js application, including the route you wish to test;
2. For example, if you want to test the GET homepage route, enter `http://localhost:3000/`.

Step 7: Send the Request

Click the "Send" button to send the request to your Express.js server.

Step 8: View Response

Postman will display the response from your server in the lower part of the window. Here, you can inspect the status code, response body, headers, and more.

Step 9: Test Other Routes

Create additional requests in Postman to test the other routes you've defined in your Express.js application. You can test POST, PUT, DELETE, and routes with URL parameters.

Step 10: Testing with Parameters

For routes that use URL parameters, you can include them in the URL in Postman. For instance, to test the `route /users/123`, enter `http://localhost:3000/users/123`.

Step 11: Inspect and Validate Responses

Carefully examine the responses you receive in Postman to ensure they match the expected behavior of your Express.js routes.

Step 12: Save Requests

Consider saving your requests in Postman collections for easy access and sharing with team members.

By mastering Postman, you'll streamline the process of testing and validating your Express.js APIs, ensuring they function flawlessly.

📖 **Introduction to Middleware in Express.js**

🤔 Understanding Middleware

Middleware allows us to process requests before they reach route handlers. It acts as a filter for incoming requests, providing a way to perform various tasks in the request-response cycle. Middleware functions take three arguments: a request object (`req`), a response object (`res`), and a `next` function, which is used to pass control to the next middleware in the pipeline.

Imagine a water pipe through which water flows. Water is pumped through one end of the pipe and passes through pressure gauges and valves, our middleware, before reaching its destination - our glass. The critical point of this analogy is that the order of these pressure gauges and valves matters.

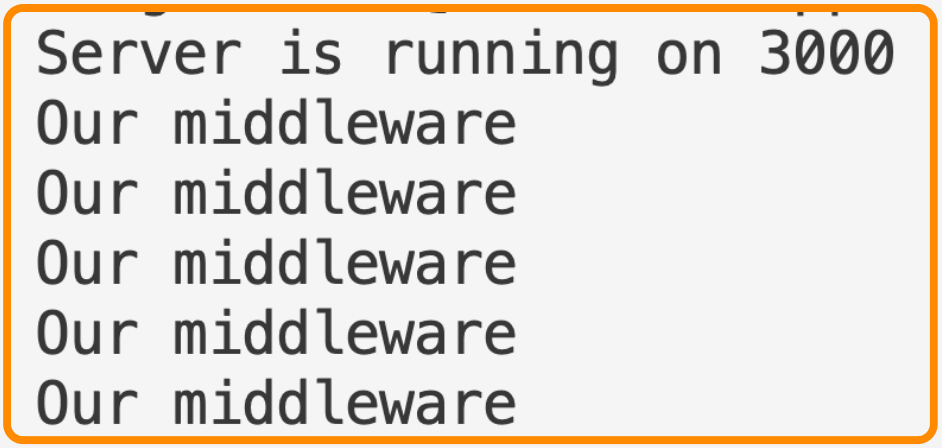
In the same way, middleware functions in Express.js are executed in a specific order, making the order of middleware registration crucial to the functionality of our application.

🏃 Middleware in Action

Let's embed our own middleware in our app before calling any route.

```
app.use((req, res, next) => {
  console.log('Our middleware');
  next();
});
```

This function does nothing, it just passes the stream through itself, but our message will always pop up in the console.



This function doesn't perform any specific task; it merely passes the stream through itself. However, it serves to illustrate how middleware works in the Express.js pipeline. In this example, whenever a request is made to our Express.js application, `Our middleware` will be logged to the console.

🛤️ Middleware Purpose

Middleware can serve various purposes in an Express.js application, including: - **Logging**: Middleware can log request details like the HTTP method, URL, and timestamp, providing insight into the traffic your server is handling; - **Authentication**: Middleware can check if a user is authenticated before allowing access to certain routes. This is often used to protect sensitive parts of your application; - **Validation**: Middleware can validate request data before processing it. For example, it can check if the data sent in a POST request is in the correct format; - **Error Handling**: Middleware can capture and handle errors that occur during request processing. This ensures that your application doesn't crash when unexpected issues arise; - **CORS (Cross-Origin Resource Sharing)**: Middleware can add CORS headers to responses, allowing or denying requests from different domains. This is essential for securing your APIs and allowing access from web pages hosted on different origins.

codefinity

🕮 Using Built-in Middleware in Express.js

In Express.js, you can access a set of built-in middleware functions designed to simplify everyday tasks in web development. These middleware functions can significantly streamline processes like parsing incoming data and serving static files. Here are some key built-in middleware functions:

express.json()

The `express.json()` middleware is used to parse incoming JSON data from requests with a JSON payload. It automatically parses the JSON data and makes it accessible through the `req.body` property for further processing.

```
const express = require('express');
const app = express();

app.use(express.json()); // Parse incoming JSON data.

app.post('/api/users', (req, res) => {
  const newUser = req.body; // Access the parsed JSON data.
  // Implement user creation logic here.
  res.send('User created.');
});
```

express.urlencoded()

The `express.urlencoded()` middleware parses incoming URL-encoded data from forms submitted via POST requests. It adds the parsed data to the `req.body` property.

```
const express = require('express');
const app = express();

app.use(express.urlencoded({ extended: true })); // Parse URL-encoded data.

app.post('/api/login', (req, res) => {
  const formData = req.body; // Access the parsed form data.
  // Validate and process login data here.
  res.send('Login successful.');
});
```

> **Note**
>
> The `{ extended: true }` option allows for handling more complex data in form submissions.

express.static()

The `express.static()` middleware serves static files, such as HTML, CSS, JavaScript, and images, from a specified directory. It's a valuable tool for serving assets like stylesheets and client-side scripts.

```
const express = require('express');
const app = express();

// Serve static files from the `public` directory.
app.use(express.static('public'));

// Now, files in the `public` directory are accessible via their URLs,
like `/styles.css`.
```

Utilizing these built-in middleware functions allows you to streamline the process of handling data and serving static files in your Express.js applications.

codefinity

📖 Creating Custom Middleware

In Express.js, we can create custom middleware functions using the `app.use()` method. These functions take three parameters: `req` (the request object), `res` (the response object), and `next` (a function to pass control to the next middleware in the chain). Custom middleware can be applied globally to all routes or specific routes by specifying a route path.

Creating Custom Middleware

Here's a basic example of a custom middleware function that logs the timestamp and URL of every incoming request:

```
app.use((req, res, next) => {
  const timestamp = new Date().toISOString();
  const url = req.url;

  console.log(`[timestamp]{timestamp}] timestamp]{url}`);

  next(); // Pass control to the next middleware.
});
```

In this example, the custom middleware logs the timestamp and URL of each incoming request to the console. The `next()` function is called to pass control to the next middleware in the pipeline.

Real-World Example: Combining Built-in and Custom Middleware

Here's a practical example where we use an Express built-in middleware to parse JSON data and then create a custom middleware to validate that data before proceeding:

codefinity

```
const express = require('express');
const app = express();

// Use built-in middleware to parse JSON data.
app.use(express.json());

// Custom middleware to validate JSON data.
app.use((req, res, next) => {
  const jsonData = req.body;

  if (!jsonData || Object.keys(jsonData).length === 0) {
    // If the JSON data is missing or empty, send an error response.
    return res.status(400).json({ error: 'Invalid JSON data' });
  }

  // Data is valid; proceed to the next middleware or route.
  next();
});

// Route that expects valid JSON data.
app.post('/api/data', (req, res) => {
  const jsonData = req.body;
  res.status(200).json({ message: 'Data received', data: jsonData });
});

app.listen(3000, () => {
  console.log('Server is running on port 3000.');
});
```

In this example: - We use the built-in middleware `express.json()` to parse incoming JSON data; - We create a custom middleware that checks if the parsed JSON data is missing or empty. If it is, it sends a 400 Bad Request response with an error message. If the data is valid, it calls `next()` to proceed to the route; - The `/api/data` route expects valid JSON data. If the custom middleware validates the data, the route handler receives it and sends a success response.

▶ Code Description

This example demonstrates how we can use both built-in and custom middleware to validate data before it reaches our route handlers, ensuring the data is in the expected format before processing it further.

codefinity

🕮 Summary of Key Express.js Concepts

We've covered the essentials of Express.js

Why use Express.js?

We learned its role, benefits over plain Node.js, and various use cases.

Setting up an Express application

We installed Express.js, created a simple app, and explored its basic structure.

Creating routes and handling requests

We mastered route creation, handling different HTTP methods, and managing URL parameters.

Middleware in Express

We understood middleware, created custom ones, used built-in middleware, and discussed execution order.

codefinity

Building REST APIs with Node.js and Express.js

codefinity

📖 Introduction to REST API Development

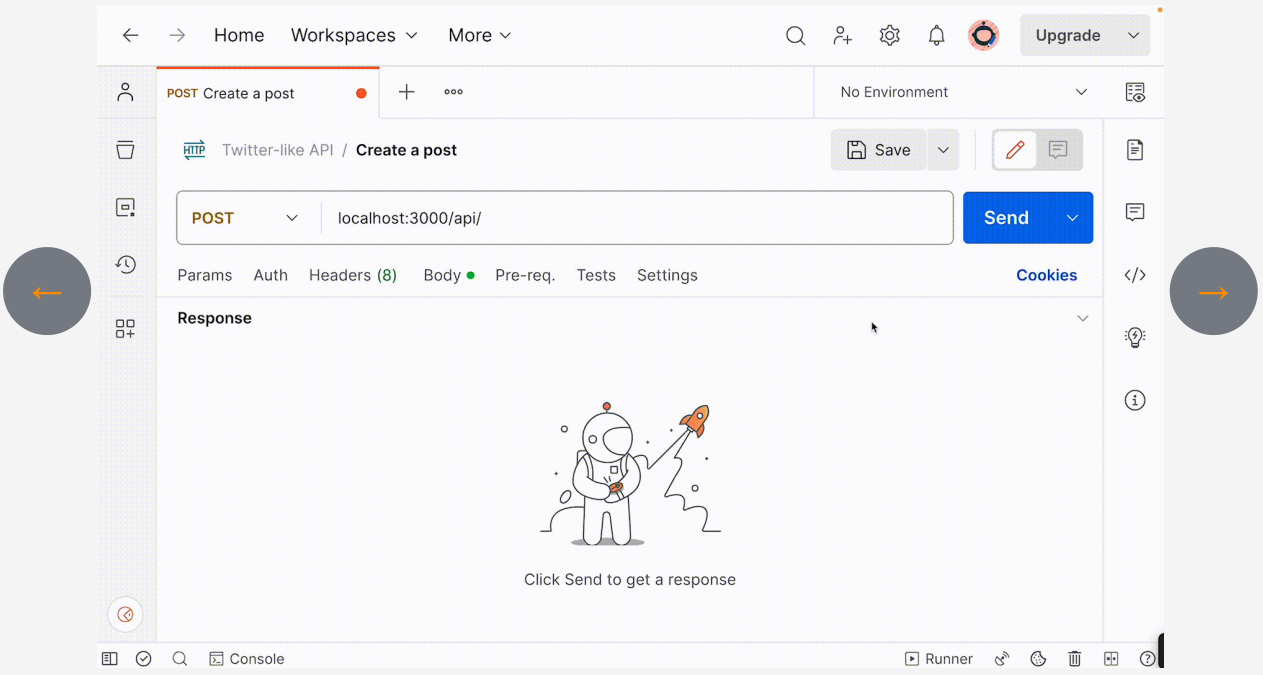
Welcome to the practical section, where we will build a Twitter-like REST API from scratch. This project serves a dual purpose: You will have the opportunity to revise everything you've learned in this course and create a real-world application to add to your portfolio.

🎯 The Purpose of the API

We will construct a simplified social networking platform that mimics the core functionalities of Twitter. By the end of this project, your app will boast the following features: - **Create New Posts**: Users can compose and publish new posts or tweets; - **Update Posts**: It's possible to make edits or alterations to existing posts; - **Read All Posts**: Users can browse and view all the posts in the app; - **Read Specific Posts**: Access a specific post by searching for it using a unique identifier; - **Delete Posts**: If a post was created mistakenly or needs removal, users can delete it.

📱 The Full App Experience

Check out the interactive slider below to glimpse the magic we'll create together. Navigate using the arrows on both the right and left sides to explore each operation individually.



🏆 What You'll Achieve

By the end of this project, you'll not only have a functional Twitter-like API under your belt, but you'll also gain valuable experience and have an impressive addition to your developer portfolio. This hands-on experience will help solidify your understanding of web development concepts and equip you with a tangible project to showcase to potential employers or clients.

codefinity

📖 Core Concepts of REST APIs

Let's dive deeper into what a REST API is and how it works, since we are about to build it. Understanding these fundamental concepts will lay a solid foundation for the rest of our project.

Plan

- 🤔 What is a REST API?
- 🔍 Key Principles of REST;
- 🤷‍♂️ How REST APIs Operate.

🤔 What is a REST API?

REST, or Representational State Transfer, is an architectural style for designing networked applications. REST APIs are a set of rules for creating and interacting with web services, facilitating seamless data exchange and operations across software systems.

🔍 Key Principles of REST

To grasp the essence of REST APIs, it's essential to remember these core principles: - **Statelessness**: In REST, each client and server interaction is self-contained. All the necessary information must be included in the request itself; - **Resource-Centric**: REST treats everything as a resource, uniquely identifying each resource by a URI (Uniform Resource Identifier). These resources interact through standard HTTP methods like GET, POST, PUT, and DELETE; - **Representation**: Resources in REST can have multiple representations, such as JSON or XML. This flexibility allows clients to choose their preferred format for data exchange.

codefinity

🤦 How REST APIs Operate

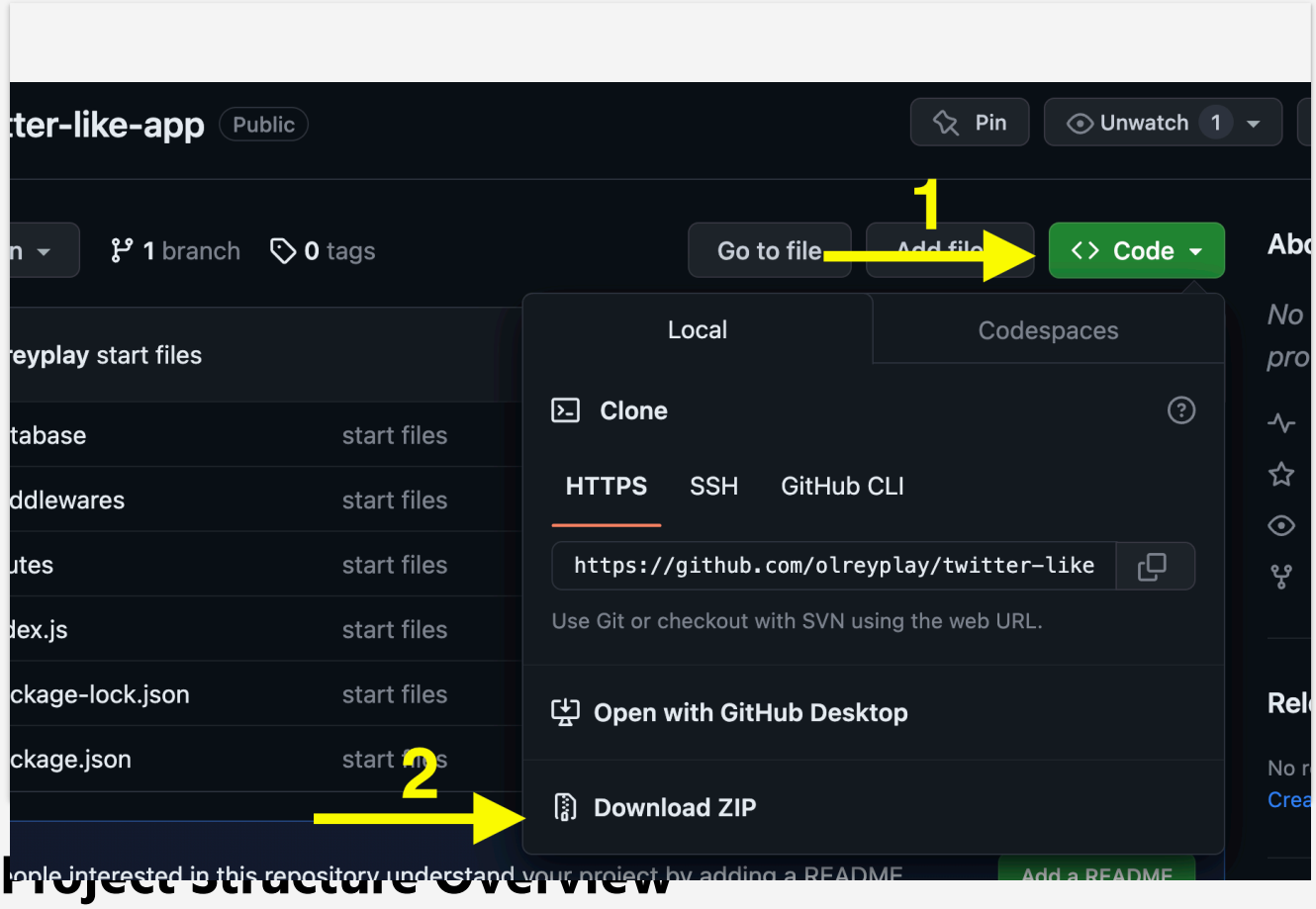
Now, let's take a closer look at how REST APIs operate. The process involves several key steps: 1. **Request**: Clients initiate requests by providing all necessary details within each request. This includes the HTTP method, URI, and required parameters or data; 2. **Resource**: The API processes the request based on the resource's unique URI. This URI serves as the address for the requested resource; 3. **HTTP Methods**: Standard HTTP methods determine the operation type to perform. For example, GET is used for retrieval, POST for creation, PUT for updating, and DELETE for removal; 4. **Response**: After processing the request, the server sends a response in the chosen representation format, typically JSON or XML. This response contains the requested data or confirmation of the action performed; 5. **Statelessness**: REST interactions are designed to be stateless, meaning no session data is stored between requests. Each request is independent and self-sufficient.

📖 Setting Up the Project Structure

In this chapter, we'll take a closer look at the project's structure. Maintaining a well-organized structure becomes crucial as we build the entire application. Below, we'll outline the key directories and files we'll encounter in our project:

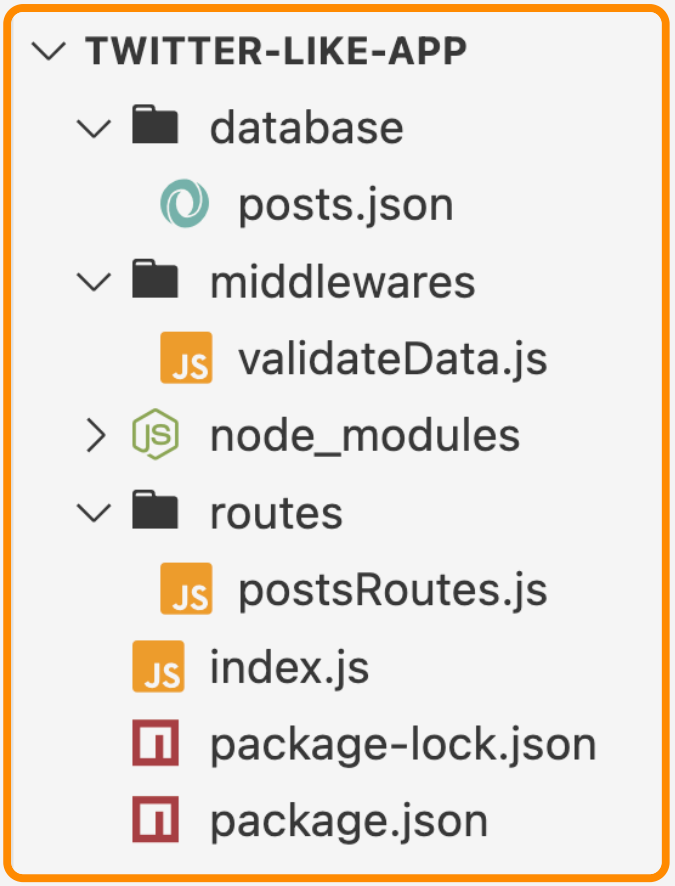
Project Initial Files

You can access the initial project files and folders on Github If you're new to GitHub, follow the simple two-step process illustrated below to download the project.



Project Structure Overview

Let's explore the purpose of each directory and file. The image below provides a visual representation of the project's structure:



Now, let's delve into the purpose of each directory and file:

1. **index.js**: This serves as the application's main entry point. Within this file, we set up the Express server, configure middleware, define routes, and initiate the server;
2. Initialization of the Express application;
3. Configuration of middleware, such as `express.json()`, for JSON request parsing;
4. Route definition and error-handling middleware;
5. Starting the Express server on a specified port (e.g., `3000`).
6. **routes/**: This directory contains route definitions for various API endpoints. Organizing routes into separate modules helps keep the codebase clean;
7. Creating distinct route files for different functionalities (e.g., user management, tweets, posts, authentication);
8. Organization and modularization of route handling code.
9. **routes/postsRoutes.js**: Specifically handling routes related to posts (tweets) within the application;
10. Definition of routes for creating, retrieving, updating, and deleting posts (tweets);
11. Management of interactions with the `posts.json` data file.
12. **middlewares/**: Middleware functions stored in this directory are essential for various tasks like validation, authentication, and authorization. They promote code reusability;

13. Segregation of middleware functions into individual modules;

14. Use of middleware for tasks such as data validation, user authentication, and error handling.

15. **middlewares/validateData.js**: This middleware function focuses on data validation within incoming requests. It ensures that the submitted data meets the required criteria;

16. Examination of incoming data for correctness before processing;

17. Appropriate error responses in cases of invalid or missing data.

18. **database/**: This directory houses the data storage for the application;

19. **database/posts.json**: In this file, we store our data - in our case, posts - in JSON format;

20. **node_modules/**: Automatically generated when we run `npm i express`, this directory contains all the external libraries and modules utilized in the project;

21. **package.json** and **package-lock.json**: These files list all the packages upon which the project depends.

codefinity

📖 Defining the Entry Point of the API

The `index.js` file is where we configure our server, define middleware, set up routes, and handle errors. It serves as the heart of our Express application.

Import required modules and files

`index.js` is where we configure our server, define middleware, set up routes, and handle errors. Let's break down the code step by step.

```
const express = require("express"); // Import the `Express` framework
const app = express(); // Create an `Express` application instance
const router = require("./routes/postsRoutes"); // Import the `router` module for posts
const PORT = process.env.PORT || 3000; // Define the port for the server
```

Middleware for JSON Parsing

```
app.use(express.json()); // Use the `express.json()` middleware for parsing JSON requests
```

The `express.json()` middleware parses incoming JSON requests and makes the data available in `req.body`. It's crucial for handling JSON data in our POST and PUT requests.

Route Setup

Routing defines how our application responds to client requests.

```
app.use("/api", router); // Use the router for handling routes under the `"/api"` path
```

Routing defines how our application responds to client requests. In this line of code, we specify that the `router` defined in `postsRoutes.js` should handle routes under the `/api` path.

Error Handling Middleware

Error handling is crucial to ensure our application handles errors gracefully.

```
// Error handling middleware
app.use((err, req, res, next) => {
  console.error(err.stack); // Log the error to the console
  res.status(500).json({ error: "Internal Server Error" }); // Send a 500 Internal Server Error response
});
```

- This middleware is responsible for catching errors that occur during request processing. If any middleware or route handler before it calls `next(err)`, this middleware will catch the error;
- It logs the error to the console using `console.error(err.stack)`;
- It sends a 500 Internal Server Error response to the client, indicating that something went wrong on the server.

Starting the Server

To complete our application setup, we start the Express server on a specified port.

```
// Start the Express server
app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`); // Log a message
indicating the server is running
});
```

- This line starts the Express server and makes it listen on the specified port (`PORT`);
- When the server starts, it logs a message to the console indicating the port on which it's running.

Complete code of the index.js file

codefinity

```
// Import required modules and files
const express = require("express"); // Import the `Express` framework
const app = express(); // Create an `Express` application instance
const router = require("./routes/postsRoutes"); // Import the `router`
module for posts
const PORT = process.env.PORT || 3000; // Define the port for the
server

app.use(express.json()); // Use the `express.json()` middleware for
parsing JSON requests

app.use("/api", router); // Use the `router` for handling routes under
the `"/api"` path

// Error handling middleware
app.use((err, req, res, next) => {
  console.error(err.stack); // Log the error to the console
  res.status(500).json({ error: "Internal Server Error" }); // Send a
`500 Internal Server Error` response
});

// Start the Express server
app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`); // Log a message
indicating the server is running
});
```

codefinity

📖 Building the GET All Posts Endpoint

We'll explore how to implement the "GET ALL POSTS" route in the `postsRoutes.js` file. This route retrieves a list of all posts from the data source (`database/posts.json`) and sends them as a response to the client.

Importing Required Modules and Dependencies

At the beginning of the file, we import the necessary modules and dependencies:

```
const express = require("express");
const fs = require("fs/promises");
const validatePostData = require("../middlewares/validateData");
```

- `express`: We import the Express framework for building our routes;
- `fs/promises`: This module provides asynchronous file operations, which we'll use to read data from a JSON file;
- `validatePostData`: Although not used in this route, we import the `validatePostData` middleware, which will be helpful for data validation in later chapters.

Initializing an Express Router

We initialize an instance of an Express router, which will handle all routes defined within this file:

```
const router = express.Router();
```

Creating a Function to Read Data

We define an asynchronous function named readData to read data from a JSON file. This function ensures that data is retrieved properly and handles errors:

```
// Function to read data from the JSON file
async function readData() {
  try {
    // Read the contents of the `posts.json` file
    const data = await fs.readFile("./database/posts.json");
    // Parse the JSON data into a JavaScript object
    return JSON.parse(data);
  } catch (error) {
    // If an error occurs during reading or parsing, throw the error
    throw error;
  }
}
```

- `fs.readFile`: We use `fs.readFile` to read the contents of the `./database/posts.json` file;
- `JSON.parse`: The data retrieved from the file is parsed into a JavaScript object;
- Error Handling: If any errors occur during the reading or parsing process, they are caught, and the error is thrown.

Defining the "GET ALL POSTS" Route

Here's how we define the "GET ALL POSTS" route within the router:

```
// GET ALL POSTS
router.get("/", async (req, res, next) => {
  try {
    // Call the `readData` function to retrieve the list of posts
    const data = await readData();
    // Send the retrieved data as the response
    res.status(200).send(data);
  } catch (error) {
    // If an error occurs during data retrieval or sending the
response
    console.error(error.message); // Log the error to the console for
debugging
  }
});
```

Route Definition: We specify that this route handles HTTP GET requests to the root path (`/`).

Route Handler: Inside the route handler function: - We call the `readData` function to retrieve the list of posts from the JSON file; - If the data retrieval is successful, we send the retrieved data as the response using `res.send(data)` ; - If any errors occur during this process, we catch the error, log it to the console for debugging (`console.error(error.message)`), and continue.

Complete code of the postsRoutes.js file at this step

codefinity

```
const express = require("express");
const fs = require("fs/promises");
const validatePostData = require("../middlewares/validateData");

const router = express.Router();

// Function to read data from the JSON file
async function readData() {
  try {
    // Read the contents of the `posts.json` file
    const data = await fs.readFile("./database/posts.json");
    // Parse the JSON data into a JavaScript object
    return JSON.parse(data);
  } catch (error) {
    // If an error occurs during reading or parsing, throw the error
    throw error;
  }
}

// GET ALL POSTS
router.get("/", async (req, res, next) => {
  try {
    // Call the `readData` function to retrieve the list of posts
    const data = await readData();
    // Send the retrieved data as the response
    res.status(200).send(data);
  } catch (error) {
    // If an error occurs during data retrieval or sending the
response
    console.error(error.message); // Log the error to the console for
debugging
```

```
  }
});
```

codefinity

📖 Building the GET Post by Id Endpoint

We will explore implementing the "GET POST BY ID" route within the `postsRoutes.js file`. This route fetches and returns a specific post based on its unique identifier (`id`) provided as part of the URL.

> **Note**
>
> The term 'database' specifically pertains to the `posts.json` file located in the `database` folder.

Route Definition

The code below defines the "GET POST BY ID" route using `router.get()`:

```
router.get("/post/:id", async (req, res, next) => { ... }
```

- This route is configured to handle HTTP GET requests;
- The route path `/post/:id` includes a parameter `:id`, which captures the post ID from the URL.

Extracting the Post ID

We extract the post ID from the request parameters using `req.params.id`:

```
const postId = req.params.id;
```

This line captures the `:id` value from the URL, making it available for further processing.

Finding the Post in the Database

Next, we search for the post with the matching ID in the database:

```
const data = await readData();

const post = data.find((post) => post.id === postId);
```

- We use the asynchronous `readData` function to retrieve data from the JSON file;
- The `find()` method is employed to locate a post with a matching ID within the retrieved data;
- The `post` variable holds the found post or `undefined` if no match is found.

Handling the Response

We handle the response based on whether a post was found or not:

```
if (!post) {
  res.status(404).json({ error: "Post not found" });
} else {
  res.status(200).send(post);
}
```

- If no post was found (i.e., `post` is `undefined`), we send a 404 response along with an error message, indicating that the requested post was not found;
- If a post was found, we send the post as the response with a status code of 200 (OK).

Error Handling

We wrap the route code in a try-catch block to handle potential errors during data retrieval or request processing. Any errors that occur are logged to the console for debugging purposes:

```
try {
  // ... (code for retrieving and processing data)
} catch (error) {
  console.error(error.message);
}
```

Complete code of the postsRoutes.js file at this step

```
const express = require("express");
const fs = require("fs/promises");
const validatePostData = require("../middlewares/validateData");

const router = express.Router();

// Function to read data from the JSON file
async function readData() {
  try {
    // Read the contents of the `posts.json` file
    const data = await fs.readFile("./database/posts.json");
    // Parse the JSON data into a JavaScript object
    return JSON.parse(data);
  } catch (error) {
    // If an error occurs during reading or parsing, throw the error
    throw error;
  }
}

// GET ALL POSTS
router.get("/", async (req, res, next) => {
  try {
    // Call the `readData` function to retrieve the list of posts
    const data = await readData();
    // Send the retrieved data as the response
    res.status(200).send(data);
  } catch (error) {
    // If an error occurs during data retrieval or sending the
response
    console.error(error.message); // Log the error to the console for
debugging
  }
```

```
});

// GET POST BY ID
router.get("/post/:id", async (req, res, next) => {
  try {
    // Extract the post ID from the request parameters
    const postId = req.params.id;
    // Read data from the JSON file
    const data = await readData();

    // Find the post with the matching ID
    const post = data.find((post) => post.id === postId);

    // If the post is not found, send a 404 response
    if (!post) {
      res.status(404).json({ error: "Post not found" });
    } else {
      // If the post is found, send it as the response
      res.status(200).send(post);
    }
  } catch (error) {
    // Handle errors by logging them and sending an error response
    console.error(error.message);
  }
});
```

📖 Building the CREATE Post Endpoint

We will examine creating a new post using the "CREATE POST" route within the `postsRoutes.js` file. This route is responsible for handling the creation of a post and saving it to the data source (`database/posts.json`).

Route Definition

The code below defines the "CREATE POST" route using `router.post()`:

```
router.post("/", validatePostData, async (req, res, next) => { ... }
```

- This route is configured to handle HTTP POST requests to the root path `/`;
- It utilizes the `validatePostData` middleware, which ensures that the data sent in the request body is valid before proceeding.

Data Validation Middleware

Before delving into the route's logic, we must create a data validation middleware. This middleware ensures that the data sent in the request body is valid before attempting to create a new post.

```
// Middleware to validate post data
function validatePostData(req, res, next) {
  const { username, postTitle, postContent } = req.body;

  if (!username || !postTitle || !postContent) {
    return res.status(400).json({ error: "Missing required fields" });
  }

  // If data is valid, proceed to the next route handler
  next();
}
```

- The `validatePostData` function extracts the `username`, `postTitle`, and `postContent` from the request body;
- It checks whether these fields are present. If any of them are missing (`!username`, `!postTitle`, or `!postContent`), it responds with a 400 (Bad Request) status code and an error message indicating missing required fields;
- If the data is valid, the middleware calls `next()`, allowing the request to continue to the route handler (the "CREATE POST" route in this case).

With the data validation middleware in place, let's continue with the "CREATE POST" route logic.

New Post Data

To create a new post, we generate a unique ID using `Date.now().toString()`. Additionally, we extract the `username`, `postTitle`, and `postContent` from the request body.

```
const newPost = {
  id: Date.now().toString(),
  username: req.body.username,
  postTitle: req.body.postTitle,
  postContent: req.body.postContent,
};
```

Add the New Post to the Database

The following steps detail how the new post is added to the database:

```
const data = await readData();
data.push(newPost);
await fs.writeFile("./database/posts.json", JSON.stringify(data));
```

- The existing data is read from the JSON file using the asynchronous `readData` function, as previously explained;
- The new post is added to the `data` array;
- The updated `data` array is then written back to the JSON file to save the newly created post.

Sending a Response

Upon successful creation of the new post, a success response is sent to the client. The response includes a status code of 201 (Created) and the details of the newly created post in JSON format.

```
res.status(201).json(newPost);
```

Error Handling

We wrap the route code in a try-catch block to handle potential errors during data retrieval or request processing. Any errors that occur are logged to the console for debugging purposes:

```
try {
  // ... (code for retrieving and processing data)
} catch (error) {
  console.error(error.message);
}
```

Complete code of the postsRoutes.js file at this step

```
const express = require("express");
const fs = require("fs/promises");
const validatePostData = require("../middlewares/validateData");

const router = express.Router();

// Function to read data from the JSON file
async function readData() {
  try {
    // Read the contents of the `posts.json` file
    const data = await fs.readFile("./database/posts.json");
    // Parse the JSON data into a JavaScript object
    return JSON.parse(data);
  } catch (error) {
    // If an error occurs during reading or parsing, throw the error
    throw error;
  }
}

// GET ALL POSTS
router.get("/", async (req, res, next) => {
  try {
    // Call the `readData` function to retrieve the list of posts
    const data = await readData();
    // Send the retrieved data as the response
    res.status(200).send(data);
  } catch (error) {
    // If an error occurs during data retrieval or sending the
response
    console.error(error.message); // Log the error to the console for
debugging
  }
```

```
});

// GET POST BY ID
router.get("/post/:id", async (req, res, next) => {
  try {
    // Extract the post ID from the request parameters
    const postId = req.params.id;
    // Read data from the JSON file
    const data = await readData();

    // Find the post with the matching ID
    const post = data.find((post) => post.id === postId);

    // If the post is not found, send a 404 response
    if (!post) {
      res.status(404).json({ error: "Post not found" });
    } else {
      // If the post is found, send it as the response
      res.status(200).send(post);
    }
  } catch (error) {
    // Handle errors by logging them and sending an error response
    console.error(error.message);
  }
});

// CREATE POST
router.post("/", validatePostData, async (req, res, next) => {
  try {
    // Generate a unique ID for the new post
    const newPost = {
      id: Date.now().toString(),
      username: req.body.username,
```

```
      postTitle: req.body.postTitle,
      postContent: req.body.postContent,
    };

    // Read the existing data
    const data = await readData();

    // Add the new post to the data
    data.push(newPost);

    // Write the updated data back to the JSON file
    await fs.writeFile("./database/posts.json", JSON.stringify(data));

    // Send a success response with the new post
    res.status(201).json(newPost);
  } catch (error) {
    // Handle errors by logging them to the console
    console.error(error.message);
  }
});
```

Complete code of the validateData.js file

```
// Middleware to validate post data
function validatePostData(req, res, next) {
  const { username, postTitle, postContent } = req.body;

  if (!username || !postTitle || !postContent) {
    return res.status(400).json({ error: "Missing required fields" });
  }

  // If data is valid, proceed to the next route handler
  next();
}

module.exports = validatePostData;
```

🕮 Building the UPDATE Post by Id Endpoint

We will research how to update an existing post using the "UPDATE POST BY ID" route within the `postsRoutes.js` file. This route handles the updating of a post based on its unique ID.

Route Definition

The code below defines the "UPDATE POST BY ID" route using `router.put()`:

```
router.put("/post/:id", validatePostData, async (req, res, next) => {
... }
```

- This route is configured to handle HTTP PUT requests, specifically for updating posts;
- It includes a parameterized `:id` in the route path to identify the post to be updated;
- The `validatePostData` middleware is added to ensure data validation before proceeding. The logic of the `validatePostData` middleware remains the same as in the previous step.

Obtaining Data from the Request

Here, we extract the necessary data from the request, including the post ID and the updated post content:

```
const postId = req.params.id;
const updatedData = {
  username: req.body.username,
  postTitle: req.body.postTitle,
  postContent: req.body.postContent,
};
```

- The post ID is extracted from the request parameters, making it available for further processing. The `:id` parameter from the route URL is captured using `req.params.id`;
- The `username`, `postTitle`, and `postContent` are extracted from the request body.

Update the Post in the Database

Updating an existing post involves several steps, as outlined below:

```
const data = await readData();

const postIndex = data.findIndex((post) => post.id === postId);

if (postIndex === -1) {
  return res.status(404).json({ error: "Post not found" });
}

data[postIndex] = {
  ...data[postIndex],
  ...updatedData,
};

await fs.writeFile("./database/posts.json", JSON.stringify(data));
```

- We read the existing data from the JSON file using the asynchronous `readData` function, as explained earlier;
- The `postIndex` variable stores the index of the post to be updated in the `data` array by comparing post IDs;
- If the post is not found (i.e., `postIndex === -1`), a 404 (Not Found) response with an error message is returned to the client;
- To update the post data, we merge the existing post data (`...data[postIndex]`) with the updated data (`...updatedData`). This ensures that only the specified fields are updated and existing data is retained;
- Finally, the updated `data` array is written back to the JSON file to save the changes made to the post.

Sending a Response

Upon successful post update, a JSON response is sent to the client. The response includes a status code of 200 (OK), indicating a successful update and the updated post data.

```
res.status(200).json(data[postIndex]);
```

Error Handling

We wrap the route code in a try-catch block to handle potential errors during data retrieval or request processing. Any errors that occur are logged to the console for debugging purposes:

```
try {
  // ... (code for retrieving and processing data)
} catch (error) {
  console.error(error.message);
}
```

Complete code of the postsRoutes.js file at this step

```
const express = require("express");
const fs = require("fs/promises");
const validatePostData = require("../middlewares/validateData");

const router = express.Router();

// Function to read data from the JSON file
async function readData() {
  try {
    // Read the contents of the `posts.json` file
    const data = await fs.readFile("./database/posts.json");
    // Parse the JSON data into a JavaScript object
    return JSON.parse(data);
  } catch (error) {
    // If an error occurs during reading or parsing, throw the error
    throw error;
  }
}

// GET ALL POSTS
router.get("/", async (req, res, next) => {
  try {
    // Call the `readData` function to retrieve the list of posts
    const data = await readData();
    // Send the retrieved data as the response
    res.status(200).send(data);
  } catch (error) {
    // If an error occurs during data retrieval or sending the
response
    console.error(error.message); // Log the error to the console for
debugging
  }
```

```
});

// GET POST BY ID
router.get("/post/:id", async (req, res, next) => {
  try {
    // Extract the post ID from the request parameters
    const postId = req.params.id;
    // Read data from the JSON file
    const data = await readData();

    // Find the post with the matching ID
    const post = data.find((post) => post.id === postId);

    // If the post is not found, send a 404 response
    if (!post) {
      res.status(404).json({ error: "Post not found" });
    } else {
      // If the post is found, send it as the response
      res.status(200).send(post);
    }
  } catch (error) {
    // Handle errors by logging them and sending an error response
    console.error(error.message);
  }
});

// CREATE POST
router.post("/", validatePostData, async (req, res, next) => {
  try {
    const newPost = {
      id: Date.now().toString(), // Generate a unique ID for the new
post
      username: req.body.username,
```

```
      postTitle: req.body.postTitle,
      postContent: req.body.postContent,
    };

    // Read the existing data
    const data = await readData();

    // Add the new post to the data
    data.push(newPost);

    // Write the updated data back to the JSON file
    await fs.writeFile("./database/posts.json", JSON.stringify(data));

    // Send a success response with the new post
    res.status(201).json(newPost);
  } catch (error) {
    // Handle errors by logging them to the console
    console.error(error.message);
  }
});

// UPDATE POST BY ID
router.put("/post/:id", validatePostData, async (req, res, next) => {
  try {
    // Extract the post ID from the request parameters
    const postId = req.params.id;
    // Extract the updated data from the request body
    const updatedData = {
      username: req.body.username,
      postTitle: req.body.postTitle,
      postContent: req.body.postContent,
    };
```

```
    // Read the existing data
    const data = await readData();

    // Find the index of the post with the specified ID in the data
array
    const postIndex = data.findIndex((post) => post.id === postId);

    // If the post with the specified ID doesn't exist, return a 404
error
    if (postIndex === -1) {
      return res.status(404).json({ error: "Post not found" });
    }

    // Update the post data with the new data using spread syntax
    data[postIndex] = {
      ...data[postIndex], // Keep existing data
      ...updatedData, // Apply updated data
    };

    // Write the updated data back
    await fs.writeFile("./database/posts.json", JSON.stringify(data));

    // Send a success response with the updated post
    res.status(200).json(data[postIndex]);
  } catch (error) {
    console.error(error.message);
    next(error);
  }
});
```

🕮 Building the DELETE Post by Id Endpoint

We'll dive into the implementation of the "DELETE POST BY ID" route within the `postsRoutes.js` file. This route allows clients to delete a specific post by providing its unique ID.

Route Definition

The code below defines the "DELETE POST BY ID" route using `router.delete()`:

```
router.delete("/post/:id", async (req, res, next) => { ... }
```

This route handles HTTP DELETE requests with a parameterized `:id` in the route path. The `:id` parameter is used to identify the post to be deleted. We don't need extra middleware like `dataValidation` as we get all the necessary information from the URL parameter.

Extracting the Post ID

We extract the post ID from the request parameters using `req.params.id`:

```
const postId = req.params.id;
```

This line captures the `:id` value from the URL, allowing us to work with it in the subsequent code.

Delete the Post

Here's how we delete the post:

```
const data = await readData();

const postIndex = data.findIndex((post) => post.id === postId);

if (postIndex === -1) {
  return res.status(404).json({ error: "Post not found" });
}

data.splice(postIndex, 1);

await fs.writeFile("./database/posts.json", JSON.stringify(data));
```

- We begin by reading the existing data from the JSON file using the asynchronous `readData` function, as explained earlier.
- We find the index of the post to delete in the `data` array by comparing post IDs.
- If the post is not found (i.e., `postIndex === -1`), we return a 404 (Not Found) response with an error message.
- Using the `splice` method, we remove the post data from the `data` array. The `postIndex` variable determines the position of the post to delete.
- The updated `data` array, with the post removed, is then written back to the JSON file to save the changes made during the deletion.

Sending a Response

A JSON response with a status code of 200 (OK) is sent to the client, indicating a successful deletion. The response includes a message confirming that the post was deleted successfully:

```
res.status(200).json({ message: "Post deleted successfully" });
```

Error Handling

We wrap the route code in a try-catch block to handle potential errors during data retrieval or request processing. Any errors that occur are logged to the console for debugging purposes:

```
try {
  // ... (code for retrieving and processing data)
} catch (error) {
  console.error(error.message);
}
```

Complete code of the postsRoutes.js file at this step

```
const express = require("express");
const fs = require("fs/promises");
const validatePostData = require("../middlewares/validateData");

const router = express.Router();

// Function to read data from the JSON file
async function readData() {
  try {
    // Read the contents of the `posts.json` file
    const data = await fs.readFile("./database/posts.json");
    // Parse the JSON data into a JavaScript object
    return JSON.parse(data);
  } catch (error) {
    // If an error occurs during reading or parsing, throw the error
    throw error;
  }
}

// GET ALL POSTS
router.get("/", async (req, res, next) => {
  try {
    // Call the `readData` function to retrieve the list of posts
    const data = await readData();
    // Send the retrieved data as the response
    res.status(200).send(data);
  } catch (error) {
    // If an error occurs during data retrieval or sending the
response
    console.error(error.message); // Log the error to the console for
debugging
  }
```

```
});

// GET POST BY ID
router.get("/post/:id", async (req, res, next) => {
  try {
    // Extract the post ID from the request parameters
    const postId = req.params.id;
    // Read data from the JSON file
    const data = await readData();

    // Find the post with the matching ID
    const post = data.find((post) => post.id === postId);

    // If the post is not found, send a 404 response
    if (!post) {
      res.status(404).json({ error: "Post not found" });
    } else {
      // If the post is found, send it as the response
      res.status(200).send(post);
    }
  } catch (error) {
    // Handle errors by logging them and sending an error response
    console.error(error.message);
  }
});

// CREATE POST
router.post("/", validatePostData, async (req, res, next) => {
  try {
    const newPost = {
      id: Date.now().toString(), // Generate a unique ID for the new
post
      username: req.body.username,
```

```
      postTitle: req.body.postTitle,
      postContent: req.body.postContent,
    };

    // Read the existing data
    const data = await readData();

    // Add the new post to the data
    data.push(newPost);

    // Write the updated data back to the JSON file
    await fs.writeFile("./database/posts.json", JSON.stringify(data));

    // Send a success response with the new post
    res.status(201).json(newPost);
  } catch (error) {
    // Handle errors by logging them to the console
    console.error(error.message);
  }
});

// UPDATE POST BY ID
router.put("/post/:id", validatePostData, async (req, res, next) => {
  try {
    // Extract the post ID from the request parameters
    const postId = req.params.id;
    // Extract the updated data from the request body
    const updatedData = {
      username: req.body.username,
      postTitle: req.body.postTitle,
      postContent: req.body.postContent,
    };
```

```
    // Read the existing data
    const data = await readData();

    // Find the index of the post with the specified ID in the data
array
    const postIndex = data.findIndex((post) => post.id === postId);

    // If the post with the specified ID doesn't exist, return a 404
error
    if (postIndex === -1) {
      return res.status(404).json({ error: "Post not found" });
    }

    // Update the post data with the new data using spread syntax
    data[postIndex] = {
      ...data[postIndex], // Keep existing data
      ...updatedData, // Apply updated data
    };

    // Write the updated data back
    await fs.writeFile("./database/posts.json", JSON.stringify(data));

    // Send a success response with the updated post
    res.status(200).json(data[postIndex]);
  } catch (error) {
    console.error(error.message);
    next(error);
  }
});

// DELETE POST BY ID
router.delete("/post/:id", async (req, res, next) => {
  try {
```

```
    // Extract the post ID from the request parameters
    const postId = req.params.id;

    // Read the existing data
    const data = await readData();

    // Find the index of the post with the specified ID in the data
array
    const postIndex = data.findIndex((post) => post.id === postId);

    // If the post with the specified ID doesn't exist, return a 404
error
    if (postIndex === -1) {
      return res.status(404).json({ error: "Post not found" });
    }

    // Remove the post from the data array using `splice`
    data.splice(postIndex, 1);

    // Write the updated data back to the data source (e.g., a JSON
file)
    await fs.writeFile("./database/posts.json", JSON.stringify(data));

    // Send a success response with the JSON response indicating
successful deletion
    res.status(200).json({ message: "Post deleted successfully" });
  } catch (error) {
    console.error(error.message);
    next(error);
  }
});
```

```
module.exports = router;
```

🕮 Running and Testing the REST API

Now that we've completed the development of our Twitter-like API, it's time to run the application and test its functionality. To start the app, open your terminal and run the following command:

```
node index
```

Once you see the success message in the terminal, you can open Postman to observe how our app responds to client requests.

> **Note**
>
> If you ever find yourself stuck or want to dive deeper into the code, you can access the complete source code of this Twitter-like API on our 🔗 **GitHub repository**.

Testing in Postman

Let's analyze the URLs responsible for different functionalities and see how the API responds to each request.

Get All Posts

Use this request to retrieve all posts from our database. No request body or extra parameters are needed. - **Method**: GET; - **URI**: `localhost:3000/api/` ; - **Response**:

```
",
e": "Kennedy Lane",
le": "Find Inspiration Everywhere",
tent": "Creativity knows no bounds. It can be discovered in everyday moments, nature's beauty, or even the bustli
. Open your eyes and senses to the world around you, and you'll find endless inspiration for your artistic endeav


",
e": "Abbot Franks",
le": "Spreading Kindness",
tent": "Kindness is a gift that keeps on giving. Whether through small acts of generosity or larger charitable ef
ness not only brightens someone else's day but also brings immense joy and fulfillment to your own life. Share th
it ripples through the world."


",
e": "Bryan Merritt",
le": "Relationship",
tent": "Sarah Clifford poisoned Robert Thunder at the beginning of the story, and it's taken the whole story to t
```

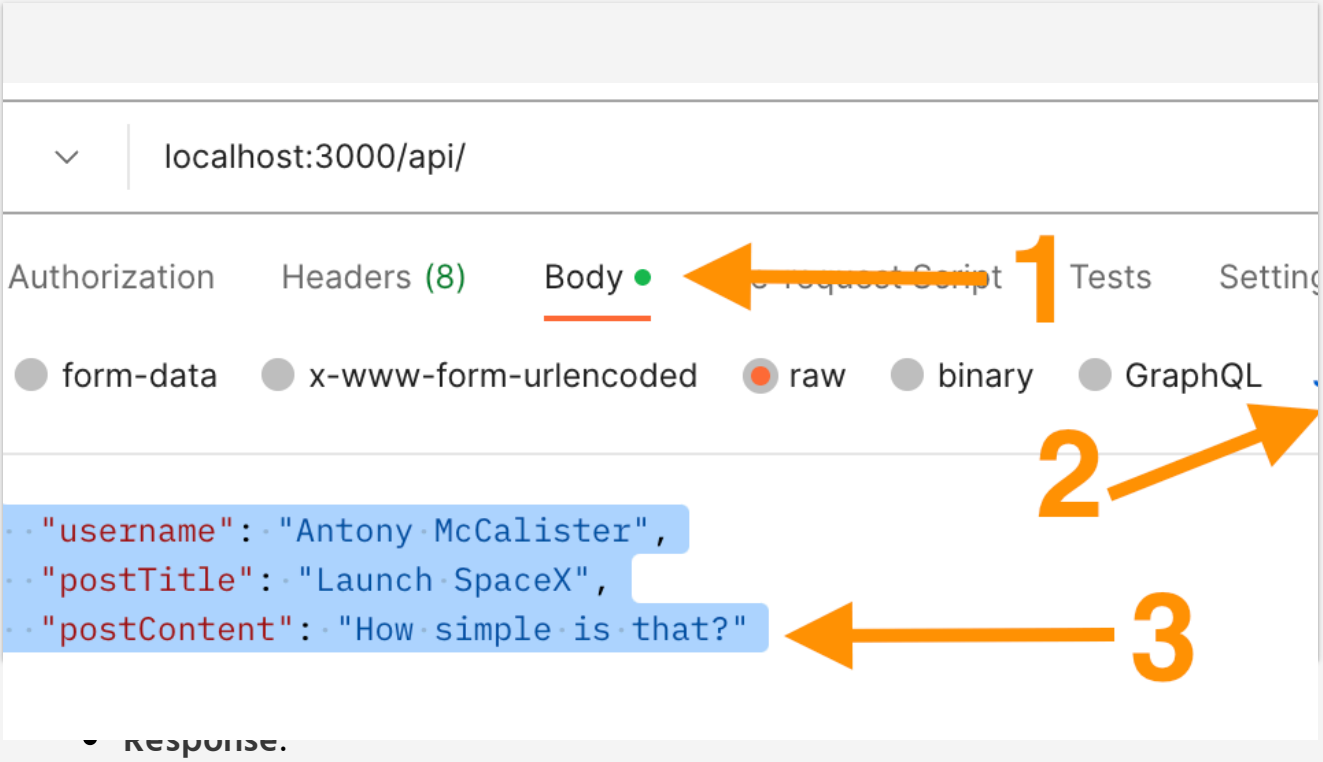
Get a Post by its ID

Retrieve a specific post based on its ID. You should pass the ID into the URL; the request body remains unchanged. - **Method**: GET; - **URI**: `localhost:3000/api/post/2`; - **Response**:

```
ess",
 gift that keeps on giving. Whether through small acts of generosity or larger c
ns someone else's day but also brings immense joy and fulfillment to your own li
rld."
```

Create a Post

Create a new post by providing valid data to the API. The data must be in JSON format and contain the correct fields. - **Method**: POST; - **URI**: `localhost:3000/api/`; - **Request Body**:

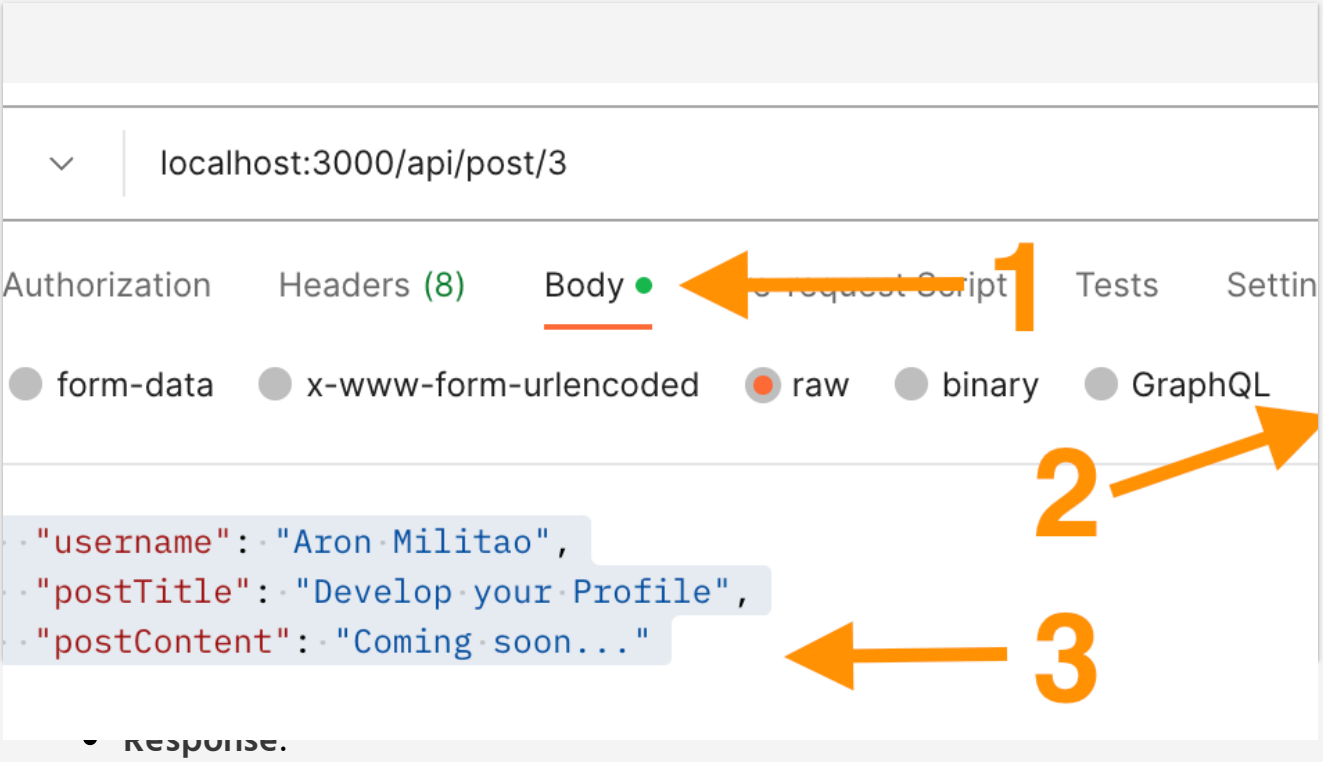


- Response:


```
{
    "id": "1694674863500",
    "username": "Antony McCalister",
    "postTitle": "Launch SpaceX",
    "postContent": "How simple is that?"
}
```


Update a Post

Update an existing post by providing the post's ID in the parameters and valid data in the request body in JSON format. The API will update the database accordingly - **Method**: PUT; - **URI**: `localhost:3000/api/post/3`; - **Request Body**:



- Response:


```
{
    "id": "3",
    "username": "Aron Militao",
    "postTitle": "Develop your Profile",
    "postContent": "Coming soon..."
}
```


Delete a Post

Delete a post from the database by passing the post ID in the URL parameters. - **Method**: DELETE; - **URI**: `localhost:3000/api/post/1`; - **Response**:

By following these steps and testing the API using Postman, you can ensure that it functions as expected, handling various requests and providing appropriate responses.

📖 Final Thoughts and Next Steps

🌟 Summary

In this extensive section, we've embarked on a journey to create a simplified Twitter-like app using Express.js and Node.js. Let's recap the key takeaways: 1. 🏗️ **Project Structure**: We started by organizing our project structure, dividing it into directories and files. This structured approach helps maintainability and scalability; 2. 🛠️ **Server Setup**: Our entry point, index.js, sets up the Express server, middleware, and routing. This is where we define the server's behavior, routes, and error handling; 3. 🛤️ **Routing**: We implemented various routes to perform CRUD (Create, Read, Update, Delete) operations on posts. Each route is carefully explained and structured for clarity and functionality; 4. 🔄 **Middleware**: We used middleware functions to validate data, ensuring that the incoming data meets specific criteria before processing it further. This enhances data integrity and security; 5. 📊 **Data Management**: We demonstrated how to read, update, and delete posts by their unique IDs, maintaining a JSON data file for storage; 6. 🔍 **Error Handling**: Effective error handling is pivotal for any application. Our error handling middleware stands ready to catch and gracefully respond to unexpected errors.

🚀 Next Steps

With a solid foundation in Node.js, Express.js, and REST API Development, you've done a fantastic job! Here are some exciting next steps to consider: - **User Authentication**: Enhance your app by adding user authentication to ensure secure access and personalized features; - **Frontend Development**: Build a user-friendly frontend interface using modern JavaScript frameworks like 📎 **React**; - **Database Integration**: Consider integrating a database like MongoDB or PostgreSQL for more efficient data storage and retrieval.